Software Requirements and Design Document

for

SYNAPSE

Prepared by: Haider Ramzan, Raja Shehryar Ameer, Hassan Rizwan

SCOPE: A comprehensive personal health ecosystem connecting medication management, vital tracking, and medical record organization.

DATE: 6th May 2026

Table of Contents

● Introduction

○ Purpose

This document specifies the software requirements for Release 1.0 of the Synapse System. It covers the complete scope of the patient-facing application, including the desktop frontend interfaces, the backend controllers, external service integrations (like storage and backup), and the central database repository.

○ Product Scope

The Synapse System is a centralized digital hub designed to empower patients to take control of their health. Its core purpose is to consolidate medication management, health logging (vitals, diet, hydration), medical record storage, and emergency preparedness into a single, cohesive platform. By automating schedules, detecting drug interactions, and providing shareable health reports, Synapse aligns with the broader healthcare goal of improving patient adherence and reducing medical errors.

○ Title :

Synapse System: Integrated Personal Healthcare Companion

○ Objectives

The primary goal of **Synapse** is to provide a comprehensive, assistive environment for health management. To achieve this, the project is guided by the following objectives:

- Centralize health tracking by uniting vitals, diet, sleep and symptoms on one platform.
- Provide digital assistance via the use of chatbot and voice recognition.
- Automate medication management by tracking inventory, prescriptions and providing notifications.
- Visualize health trends using graphs and provide weekly and monthly reports.
- Ensure data safety and organization by managing all data in one place using file management and backing up data securely.

○ Problem Statement

Currently, patients manage their health data across highly fragmented systems: paper prescriptions, separate mobile apps for diet and fitness, and scattered physical medical records. This fragmentation leads to poor medication adherence, accidental drug interactions, lost lab results, and a dangerous lack of critical health information during medical emergencies. The Synapse System mitigates these risks by providing a unified, secure, and highly cohesive automated platform for all personal health data.

● Overall Description

○ Product Perspective

The Synapse System is a new, self-contained desktop software product designed to replace the highly fragmented and manual methods of personal health tracking, such as paper prescriptions, physical medical folders, and disparate single-purpose mobile applications. It is not a follow-on member of an existing product family, but rather a standalone application built from the ground up to serve as a centralized personal health hub for the user.

While the application is self-contained, it operates by interfacing with two primary external systems to achieve its functionality. First, it interfaces with a Relational Database Management System. The application uses the Hibernate Object Relational Mapping framework to communicate with this database, which is responsible for securely persisting all structured health data, user credentials, medication inventories, and daily logs. Second, the system interfaces directly with the host operating system's local file storage. This file system interface is utilized to read uploaded medical documents and to write exported files, such as generated PDF health reports and compressed database backup files.

The overall system architecture is divided into distinct, interconnected subsystems. The front-end subsystem is driven by JavaFX, providing the graphical user interface for the patient. This interfaces with the core application logic subsystem, which contains the controllers and pure fabrication services like the Document Generator and Backup Service. Finally, the data access subsystem manages the transaction boundaries and passes information between the core logic and the external SQL database and local file system. Together, these components provide a robust, offline-capable environment for comprehensive health management.

○ Product Functions

The Synapse System provides a comprehensive suite of features organized into the following major functional groups:

User and Session Management

- Secure user registration and authentication.
- Active session tracking to ensure data privacy and access control.
- Secure logout functionality.

Health and Wellness Tracking

- Logging and monitoring of daily vitals, including blood pressure, heart rate, and body temperature.
- Tracking daily diet, weight, and hydration levels against personal goals.
- Recording specific health symptoms and maintaining a daily personal journal.

Medication Management

- Adding and updating medication details within a personal inventory.

- Tracking medicine quantities to prevent low stock.
- Setting and managing complex prescription schedules and dosages.

Emergency Preparedness

- Creating and maintaining an emergency profile containing blood type, allergies, and chronic conditions.
- Instantly compiling and displaying critical data, including active medications, during a medical emergency.

Medical Records and Scheduling

- Uploading, categorizing, and securely storing digital copies of physical medical records, such as PDFs and images.
- Creating and managing health-related calendar events and medical appointments.
- Providing a directory interface to browse external healthcare facilities.

Analytics and System Utilities

- Visualizing aggregated health data through graphical analytics dashboards.
- Generating and exporting formatted PDF health reports based on specified date ranges and categories.
- Creating compressed backup files of the entire system database and restoring data from previous backups.

○ List of Use Cases

- Manage Account
- Log Vitals
- Log Symptoms
- Track Diet, BMI and Hydration
- Write Journal Entry
- Manage Medicine Inventory and Prescriptions
- Send System Alert
- Check Drug Interaction
- Track Calendar
- Browse Hospitals
- Trigger Emergency Protocol
- Organize Medical Records
- Backup and Restore Data
- View Health Analytics
- Generate Health Report

- ## Extended Use Cases

Use Case: Manage Account

Use Case Name	Manage Account
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to create, update and manage their personal health profile securely. System – needs accurate user data to personalize health tracking and recommendations.
Preconditions	The Synapse application is installed and running on the user's desktop.
Postconditions	The patient's account is created or updated successfully and the profile data is stored in the system.
Main Success Scenario	

Actor Action	System Response
1. Patient launches the Synapse application.	
	2. System presents the login or registration options.
3. Patient chooses to register and provides personal and medical details via the onboarding interview.	
	4. System validates the provided information.
	5. System creates the account and stores the profile data.
	6. System confirms successful account creation and logs the patient in.
7. Patient reviews and optionally edits profile information.	
	8. System saves the updated profile.

Extensions	3a. Patient chooses to log in instead: Patient provides credentials; system authenticates and grants access. 4a. Validation fails (missing or invalid data): System notifies the patient and requests correction. 4b. Username or email already exists: System informs the patient and asks for an alternative.

UC-02

Use Case: Log Vitals

<table>
<tr><td>Use Case Name</td><td>Log Vitals</td></tr>
<tr><td>Scope</td><td>Synapse – Healthcare & Lifestyle Management System</td></tr>
<tr><td>Level</td><td>User-Goal</td></tr>
<tr><td>Primary Actor</td><td>Patient</td></tr>
<tr><td>Stakeholders and Interests</td><td>Patient – wants to record physiological data (blood pressure, heart rate, temperature, etc.) to monitor health trends.
System – needs vitals data to generate analytics and trigger health alerts.</td></tr>
<tr><td>Preconditions</td><td>Patient is logged into the Synapse system.</td></tr>
<tr><td>Postconditions</td><td>The vitals entry is recorded, timestamped and available for analytics.</td></tr>
<tr><td>Main Success Scenario</td><td></td></tr>
<tr><td></td><td>
<table>
<tr><th>Actor Action</th><th>System Response</th></tr>
<tr><td>1. Patient selects the Log Vitals option.</td><td></td></tr>
<tr><td></td><td>2. System presents the vitals entry form with fields for blood pressure, heart rate, temperature, etc.</td></tr>
<tr><td>3. Patient enters the current vital readings.</td><td></td></tr>
<tr><td></td><td>4. System validates the entered values against acceptable medical ranges.</td></tr>
<tr><td></td><td>5. System saves the vitals entry with a timestamp.</td></tr>
<tr><td></td><td>6. System confirms the successful log and updates the health dashboard.</td></tr>
<tr><td></td><td>7. System checks for any abnormal readings and flags warnings if necessary.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>4a. Entered values are out of acceptable range: System warns the patient and asks to verify or correct the reading.
7a. Abnormal vitals detected: System generates an alert and suggests the patient take precautionary measures.</td></tr>
</table>

UC-03

Use Case: Log Symptoms

<table>
<tr><td>Use Case Name</td><td>Log Symptoms</td></tr>
<tr><td>Scope</td><td>Synapse – Healthcare & Lifestyle Management System</td></tr>
<tr><td>Level</td><td>User-Goal</td></tr>
<tr><td>Primary Actor</td><td>Patient</td></tr>
<tr><td>Stakeholders and Interests</td><td>Patient – wants to track symptoms over time for personal awareness and doctor consultations.
System – needs symptom data to identify health patterns and generate reports.</td></tr>
<tr><td>Preconditions</td><td>Patient is logged into the Synapse system.</td></tr>
<tr><td>Postconditions</td><td>The symptom entry is recorded with severity and timestamp, and is available for trend analysis.</td></tr>
<tr><td>Main Success Scenario</td><td></td></tr>
<tr><td></td><td>
<table>
<tr><th>Actor Action</th><th>System Response</th></tr>
<tr><td>1. Patient selects the Log Symptoms option.</td><td></td></tr>
<tr><td></td><td>2. System presents the symptom logging form.</td></tr>
<tr><td>3. Patient describes the symptom and selects its severity level.</td><td></td></tr>
<tr><td>4. Patient optionally adds notes or context about the symptom.</td><td></td></tr>
<tr><td></td><td>5. System validates and saves the symptom entry with a timestamp.</td></tr>
<tr><td></td><td>6. System confirms the log and updates the symptom history.</td></tr>
<tr><td></td><td>7. System checks for recurring symptom patterns and notifies the patient if a trend is detected.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>3a. Patient cannot find the symptom in the list: Patient manually types the symptom name.
7a. Recurring pattern detected: System advises the patient to consult a healthcare professional.</td></tr>
</table>

UC-04

Use Case: Track Diet, BMI and Hydration

Use Case Name	Track Diet, BMI and Hydration
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to monitor daily caloric/water intake and BMI to maintain a healthy lifestyle. System – needs dietary data to calculate BMI, track nutrition trends, generate health analytics and trigger hydration reminders.
Preconditions	Patient is logged into the Synapse system and has a profile with height/weight information.
Postconditions	The dietary/water entries are saved, BMI is updated and reminders are scheduled.
Main Success Scenario	

Actor Action	System Response
1. Patient selects the Track Diet & Hydration option.	
	2. System presents the logging form.
3. Patient logs the food items consumed along with portion size.	
4. Patient enters daily water consumption goal (if not already provided) and logs the water intake.	
	5. System calculates the caloric intake and total consumed water volume.
6. Patient optionally updates weight information.	
	7. System recalculates the BMI based on current height and weight.
	8. System saves the dietary entry and confirms the updated BMI.
	9. System schedules hydration reminders based on daily goal.
	10. System updates the diet and BMI trends on the health dashboard.

Extensions	3a. Food item not found in the database: Patient enters custom nutritional values manually. 7a. BMI falls in an unhealthy range: System alerts the patient with dietary recommendations.

UC-05

Use Case: Write Journal Entry

<table>
<tr><td>Use Case Name</td><td>Write Journal Entry</td></tr>
<tr><td>Scope</td><td>Synapse – Healthcare & Lifestyle Management System</td></tr>
<tr><td>Level</td><td>User-Goal</td></tr>
<tr><td>Primary Actor</td><td>Patient</td></tr>
<tr><td>Stakeholders and Interests</td><td>Patient – wants to record personal thoughts, feelings and mental health observations for self-reflection.
System – needs journal data to support mental health tracking and analytics.</td></tr>
<tr><td>Preconditions</td><td>Patient is logged into the Synapse system.</td></tr>
<tr><td>Postconditions</td><td>The journal entry is saved with a timestamp and is available in the patient's journal history.</td></tr>
<tr><td>Main Success Scenario</td><td></td></tr>
<tr><td></td><td>
<table>
<tr><th>Actor Action</th><th>System Response</th></tr>
<tr><td>1. Patient selects the Write Journal Entry option.</td><td></td></tr>
<tr><td></td><td>2. System opens the journal editor.</td></tr>
<tr><td>3. Patient writes the journal entry, describing thoughts, mood or health observations.</td><td></td></tr>
<tr><td>4. Patient optionally selects a mood tag for the entry.</td><td></td></tr>
<tr><td>5. Patient submits the journal entry.</td><td></td></tr>
<tr><td></td><td>6. System saves the journal entry with a timestamp.</td></tr>
<tr><td></td><td>7. System confirms the entry is saved and updates the journal history.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>5a. Patient decides to discard the entry: System asks for confirmation before discarding unsaved content.</td></tr>
</table>

UC-06

Use Case: Manage Medicine Inventory & Prescriptions

Use Case Name	Manage Medicine Inventory & Prescriptions
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to track medicine stock levels and manage active prescriptions. System – Tracks the inventory and expiry dates, ensures adequate stock and sends reminders for medicines tagged as prescribed.
Preconditions	Patient is logged into the Synapse system.
Postconditions	The medicine inventory is updated, and the system is ready to send alerts for low stock, expiry and prescription reminders.
Main Success Scenario	

Actor Action	System Response
1. Patient selects the Manage Medicine Inventory option.	
	2. System displays the current medicine inventory.
3. Patient adds or edits a new medicine with details (name, dosage, quantity, expiry date).	
4. Patient optionally tags the medicine as prescribed and enters dosage and time.	
	5. System validates the medicine details.
	6. System saves the medicine to the inventory.
	7. System checks the inventory for low-stock or near-expiry items.
	8. If tagged as prescribed, system sets up relevant reminders.
	9. System sends alerts if any medicine is running low or nearing expiry.
	10. System confirms the inventory update.

Extensions	3a. Patient edits an existing medicine entry: System loads the entry, patient modifies details and system saves changes. 3b. Patient deletes a medicine entry: System asks for confirmation and removes the item. 5a. Invalid medicine details provided: System notifies the patient and requests correction. 8a. Drug interaction detected with existing medicines (extend: Check Drug Interaction): System warns the patient about the potential conflict.

UC-07

Use Case: Send System Alert

Use Case Name	Send System Alert
Scope	Synapse – Healthcare & Lifestyle Management System
Level	Sub-function
Primary Actor	System Timer
Stakeholders and Interests	Patient – wants to receive timely reminders for medication, appointments and health-related tasks. System – needs to proactively notify the patient based on scheduled events and health thresholds.
Preconditions	The Synapse system is running and the patient has configured alerts, medication schedules or calendar events.
Postconditions	The alert or notification is delivered to the patient via a desktop notification.
Main Success Scenario	

Actor Action	System Response
1. System Timer triggers at a scheduled time or threshold event.	
	2. System identifies the type of alert (medication reminder, appointment, low stock, etc.).
	3. System composes the notification message with relevant details.
	4. System delivers a desktop notification to the patient.
5. Patient views and acknowledges the alert.	
	6. System logs the alert delivery and acknowledgment status.

Extensions	5a. Patient dismisses the alert without action: System reschedules a follow-up reminder after a set interval. 4a. Patient is not actively using the system: System queues the notification for display upon next login.

UC-08

Use Case: Check Drug Interaction

Use Case Name	Check Drug Interaction
Scope	Synapse – Healthcare & Lifestyle Management System
Level	Sub-function
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to ensure that newly added medicines do not conflict with existing medications. System – needs to cross-reference medicines to prevent harmful drug interactions.
Preconditions	Patient is logged in and is adding or updating a medicine in the inventory. At least one other medicine already exists in the inventory.
Postconditions	The system has checked for interactions and informed the patient of any conflicts.
Main Success Scenario	

Actor Action	System Response
1. Patient adds or updates a medicine in the inventory.	
	2. System retrieves the list of all current medicines in the inventory.
	3. System cross-references the new medicine against the existing ones using the drug interaction engine.
	4. System identifies potential drug interactions, if any.
	5. System displays the interaction results to the patient.
6. Patient reviews the interaction warnings and decides whether to proceed.	
	7. System saves the patient's decision and updates the inventory accordingly.

Extensions	4a. No interactions found: System confirms the medicine is safe to add and proceeds normally. 6a. Patient decides not to add the conflicting medicine: System cancels the addition and retains the current inventory.

UC-09

Use Case: Track Calendar

<table>
<tr><td>Use Case Name</td><td>Track Calendar</td></tr>
<tr><td>Scope</td><td>Synapse – Healthcare & Lifestyle Management System</td></tr>
<tr><td>Level</td><td>User-Goal</td></tr>
<tr><td>Primary Actor</td><td>Patient</td></tr>
<tr><td>Stakeholders and Interests</td><td>Patient – wants to schedule and track medical appointments, medication times and health-related tasks.
System – needs calendar data to trigger timely alerts and reminders.</td></tr>
<tr><td>Preconditions</td><td>Patient is logged into the Synapse system.</td></tr>
<tr><td>Postconditions</td><td>The calendar event is saved and the system is configured to send reminders at the scheduled time.</td></tr>
<tr><td>Main Success Scenario</td><td></td></tr>
<tr><td></td><td>
<table>
<tr><th>Actor Action</th><th>System Response</th></tr>
<tr><td>1. Patient selects the Track Calendar option.</td><td></td></tr>
<tr><td></td><td>2. System displays the patient's calendar with existing events.</td></tr>
<tr><td>3. Patient creates a new event (appointment, medication reminder, follow-up, etc.).</td><td></td></tr>
<tr><td>4. Patient sets the date, time and reminder preferences for the event.</td><td></td></tr>
<tr><td></td><td>5. System validates the event details and checks for scheduling conflicts.</td></tr>
<tr><td></td><td>6. System saves the event and schedules the corresponding alert.</td></tr>
<tr><td></td><td>7. System confirms the event creation and displays it on the calendar.</td></tr>
</table>
</td></tr>
<tr><td>Extensions</td><td>3a. Patient edits an existing event: System loads the event, patient modifies details and system saves the updates.
3b. Patient deletes an existing event: System asks for confirmation and removes the event and its associated alerts.
5a. Scheduling conflict detected: System notifies the patient and suggests alternative times.</td></tr>
</table>

UC-10

Use Case: Browse Hospitals

Use Case Name	Browse Hospitals
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to find nearby hospitals, clinics or pharmacies for medical needs. Hospital DB – provides hospital directory data. Medicine DB – provides pharmacy and medicine availability data.
Preconditions	Patient is logged into the Synapse system. The Hospital and Medicine databases are accessible.
Postconditions	The patient has browsed and retrieved relevant hospital or pharmacy information.
Main Success Scenario	

Actor Action	System Response
1. Patient selects the Browse Hospitals option.	
	2. System retrieves the hospital and pharmacy directory from the databases.
	3. System displays the list of available hospitals and pharmacies.
4. Patient searches or filters the list by location, specialty or services.	
	5. System displays the filtered results with details (name, address, contact, services).
6. Patient selects a hospital to view more details.	
	7. System displays the detailed profile of the selected hospital.

Extensions	4a. No results match the search criteria: System informs the patient and suggests broadening the search. 2a. Database is temporarily unavailable: System notifies the patient and suggests trying again later.

UC-11

Use Case: Trigger Emergency Protocol

Use Case Name	Trigger Emergency Protocol
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to quickly access and share critical medical information during an emergency. Emergency Contacts / Medical Staff – need access to the patient's critical data for rapid response.
Preconditions	Patient is logged into the Synapse system and has medical records and emergency contact information stored.
Postconditions	Critical medical information is displayed or exported and emergency contacts are notified.
Main Success Scenario	

Actor Action	System Response
1. Patient activates the Emergency Protocol.	
	2. System immediately retrieves the patient's critical medical information (allergies, medications, conditions, blood type).
	3. System displays the critical information in a clear, accessible format.
	4. System provides options to export the data as a shareable document.
5. Patient chooses to export or share the emergency data.	
	6. System generates the emergency document and makes it available for sharing.
	7. System logs the emergency protocol activation.

Extensions	2a. Critical medical data is incomplete: System displays available data and warns about missing information. 5a. Patient only views the data without exporting: System keeps the data on-screen and skips the export step.

UC-12

Use Case: Organize Medical Records

Use Case Name	Organize Medical Records
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to digitally store, organize and retrieve prescriptions, lab reports and imaging records. System – needs organized records to support ease of access to medical documents.
Preconditions	Patient is logged into the Synapse system.
Postconditions	All records are organized in appropriate folders and files are stored in their original formats.
Main Success Scenario	

Actor Action	System Response
1. Patient selects the Organize Medical Records option.	
	2. System displays the existing categories/folders (e.g. prescriptions, lab reports etc.)
3. Patient selects a category and uploads a new document in a supported format (.txt, .png, .docx etc.)	
	4. System validates the file format and size.
5. Patient assigns a title to the uploaded record.	
	6. System saves the record and makes it searchable.
	7. System confirms the upload and updates the records library.

Extensions	2a. Patient wants to manage existing records: 1. Patient selects an existing record. 2. System provides option to move, rename or delete that record. a. If deletion is selected, then the system requires a final confirmation before the deletion. 3. Patient confirms the action and updates the library. 4a. Unsupported format: System notifies the patient and provides a list of supported formats.

UC-13

Use Case: Backup and Restore Data

Use Case Name	Backup and Restore Data
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to safeguard all health data and ensure recoverability in case of data loss. System – needs to provide secure JSON-based data export and import for data portability.
Preconditions	Patient is logged into the Synapse system.
Postconditions	Data is successfully backed up to a JSON file or restored from an existing backup file.
Main Success Scenario	

Actor Action	System Response
1. Patient selects the Backup and Restore option.	
	2. System presents options to create a new backup or restore from an existing one.
3. Patient chooses to create a backup.	
	4. System compiles all user data (vitals, symptoms, diet, records, inventory, journal, calendar).
	5. System exports the data to a secure JSON file and saves it to the chosen location.
	6. System confirms the backup is created successfully.

Extensions	3a. Patient chooses to restore data instead: Patient selects a backup file; system validates and imports the data, replacing current records after confirmation. 5a. Backup file creation fails (disk full or permission error): System notifies the patient and suggests an alternative location. 3a-1. Backup file is corrupted or incompatible: System rejects the restore and informs the patient.

UC-14

Use Case: View Health Analytics

Use Case Name	View Health Analytics
Scope	Synapse – Healthcare & Lifestyle Management System
Level	User-Goal
Primary Actor	Patient
Stakeholders and Interests	Patient – wants to visualize health trends and patterns over time to make informed decisions. System – needs to aggregate health data and present it as meaningful graphs and summaries.
Preconditions	Patient is logged into the Synapse system and has logged sufficient health data (vitals, symptoms, diet, etc.).
Postconditions	The patient has viewed the analytics dashboard with health trends and insights.
Main Success Scenario	
Extensions	2a. Insufficient data to generate meaningful analytics: System informs the patient and suggests logging more data. 7a. Patient chooses to generate a health report (extend: Generate Health Report): System proceeds to the report generation use case.

Actor Action	System Response
1. Patient selects the View Health Analytics option.	
	2. System aggregates the patient's health data across all tracked categories.
	3. System generates visual graphs and trend summaries (vitals over time, diet trends, symptom frequency, etc.).
	4. System displays the analytics dashboard to the patient.
5. Patient selects a specific category or time range to filter the analytics.	
	6. System updates the graphs and summaries based on the selected filters.
7. Patient reviews the insights and trends.	

UC-15

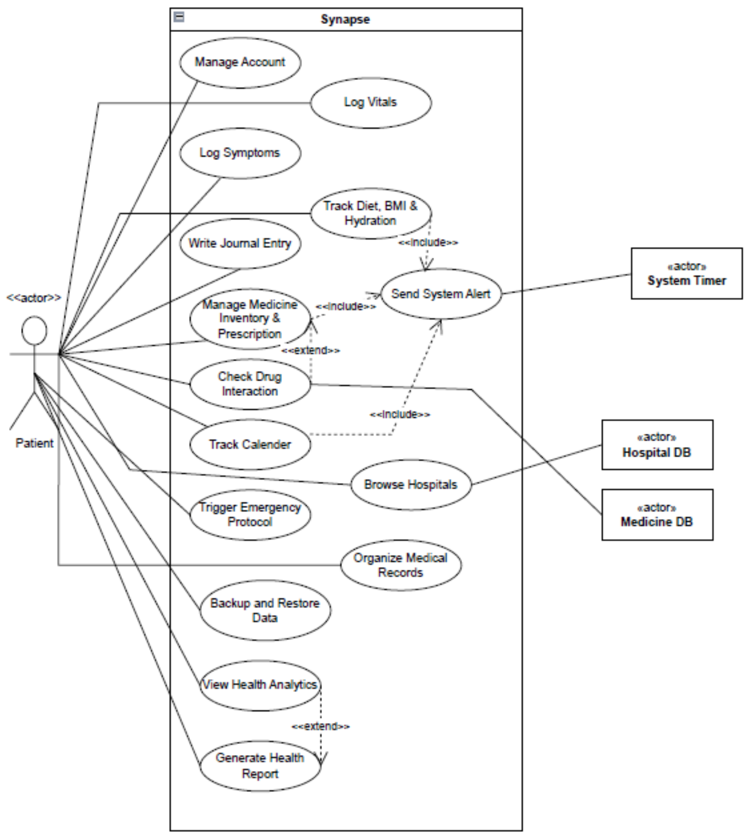
Use Case: Generate Health Report

Use Case Name	Generate Health Report
Scope	Synapse – Healthcare & Lifestyle Management System
Level	Sub-function
Primary Actor	Patient
Stakeholders and Interests	Patient – wants a professional PDF report of health data for doctor consultations or personal records. System – needs to compile health logs and analytics into a structured, exportable document.
Preconditions	Patient is logged into the Synapse system and has viewed health analytics with sufficient data.
Postconditions	A professional PDF health report is generated and saved or exported.
Main Success Scenario	

Actor Action	System Response
1. Patient selects the Generate Health Report option from the analytics view.	
	2. System presents report configuration options (date range, categories to include).
3. Patient selects the desired date range and data categories for the report.	
	4. System compiles the selected health data, graphs and summaries.
	5. System generates a professional PDF report with the compiled data.
	6. System presents the report preview to the patient.
7. Patient confirms and saves or exports the report.	
	8. System saves the report and confirms successful generation.

Extensions	4a. Selected date range has no data: System notifies the patient and suggests selecting a different range. 7a. Patient chooses to discard the report: System cancels the generation without saving.

- ## Use Case Diagram


Synapse
Manage Account
Log Vitals
Log Symptoms
Track Diet, BMI & Hydration
Write Journal Entry
Send System Alert
Manage Medicine Inventory & Prescription
Check Drug Interaction
Track Calender
Browse Hospitals
Trigger Emergency Protocol
Organize Medical Records
Backup and Restore Data
View Health Analytics
Generate Health Report
<<include>>
<<extend>>
<<actor>>
Patient
«actor» System Timer
«actor» Hospital DB
«actor» Medicine DB

● Other Nonfunctional Requirements

○ Performance Requirements

The system user interface built with JavaFX must respond to user interactions, such as navigating between different views, in less than 1 second to ensure a fluid experience. The Emergency Protocol view must compile and display critical data within 0.5 seconds, as immediate access is necessary for this life-saving feature. Furthermore, database queries and heavy operations, such as generating PDF reports or creating backup files, must execute asynchronously or resolve quickly without locking the main user interface thread. Export operations should complete within 5 seconds under normal conditions.

○ Safety Requirements

The application must prioritize data integrity and prevent system crashes that could lead to data loss. To prevent Hibernate LazyInitializationExceptions, the data access layer must correctly force-initialize necessary lazy collections, such as vital logs and medicines, before the active database session closes. Exception handling must gracefully catch database or file system errors and display user-friendly feedback via the user interface, strictly preventing silent failures. Additionally, a medical disclaimer must be displayed stating that the system's drug interaction checks and health analytics are informational and do not replace professional medical advice.

○ Security Requirements

User sessions must be strictly enforced. The SessionManager must securely track the active Patient entity, and unauthenticated users must be entirely blocked from accessing internal dashboards or data views. Passwords must be securely hashed before being stored in the database. All data stored in the underlying relational database must be managed securely through parameterized Hibernate queries to completely prevent SQL injection vulnerabilities. Local file operations, such as saving medical record PDFs, must isolate user data into secure directories.

○ Software Quality Attributes

Maintainability is a primary attribute for this system; the codebase must adhere strictly to object-oriented GRASP principles, such as Information Expert and Pure Fabrication, and utilize a strict Model View Controller architecture to ensure future developers can easily navigate and extend the system. Robustness is also critical; all operations across views must be wrapped in appropriate error-handling blocks to surface backend issues transparently. Finally, usability is highly prioritized, ensuring that the JavaFX interface is intuitive, forms are easy to navigate, and critical features like the Emergency Protocol are distinctly visible.

○ Business Rules

The primary business rule is that only an authenticated Patient account can view, create, modify, or delete their specific medical data. The system operates on a single-user-session basis; operations attempting to map entities without an active SessionManager state must be immediately rejected and prompt the user to log in. Additionally, medication inventory quantities cannot drop below zero, and the system must retain a log or history of generated health reports and backups.

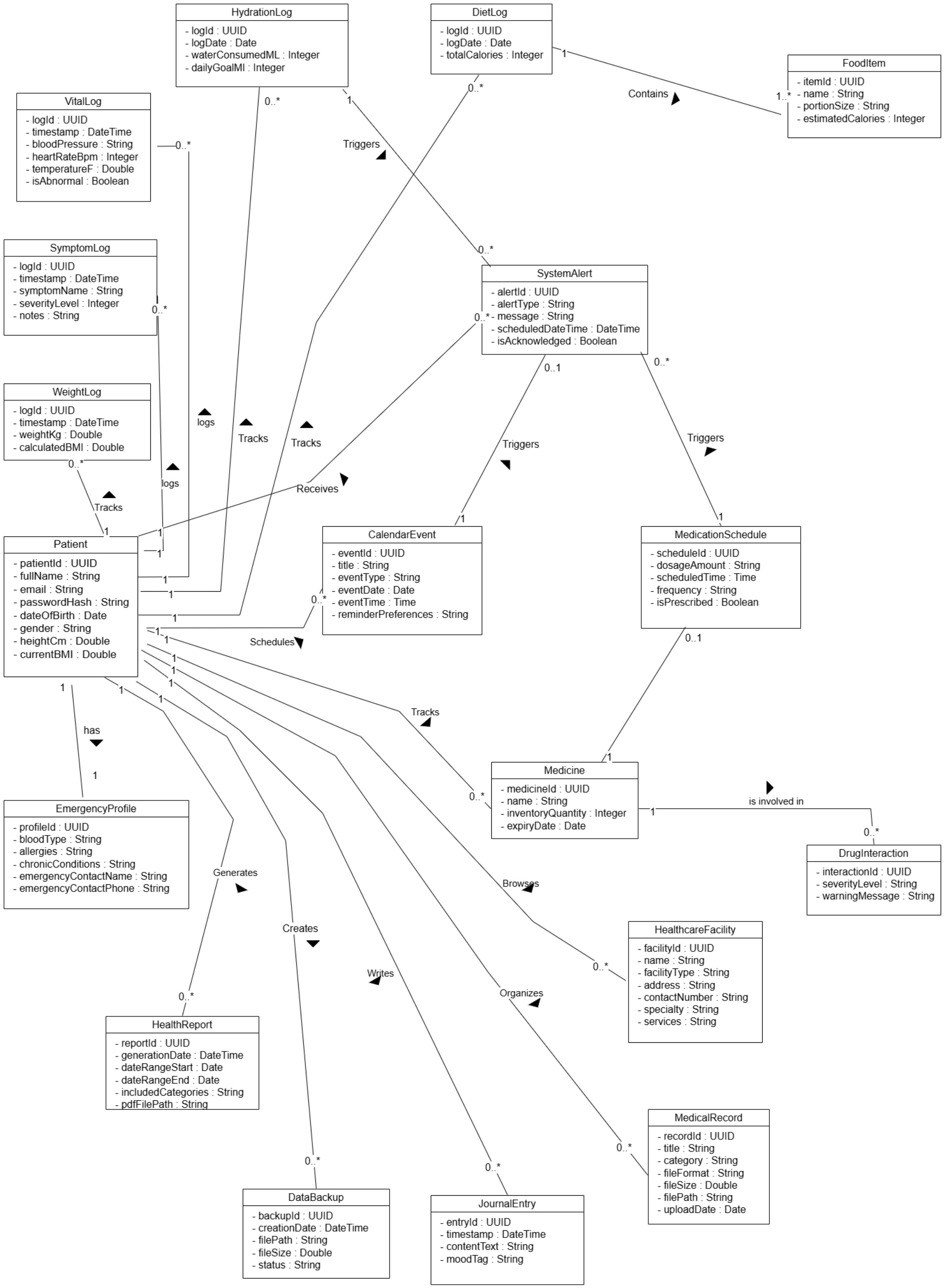
○ Operating Environment

The software is designed to operate as a standalone desktop application. It requires a hardware platform capable of running Java 17 or higher. The application is platform-agnostic and will run on Windows, macOS, or Linux environments. It relies on the JavaFX framework for rendering the graphical user interface and utilizes Hibernate ORM for database mapping. The system requires a relational database environment, such as Sql Server or a local H2 database, to handle structured data and relationship persistence, alongside standard access to the local file system for document storage and backups

○ User Interfaces

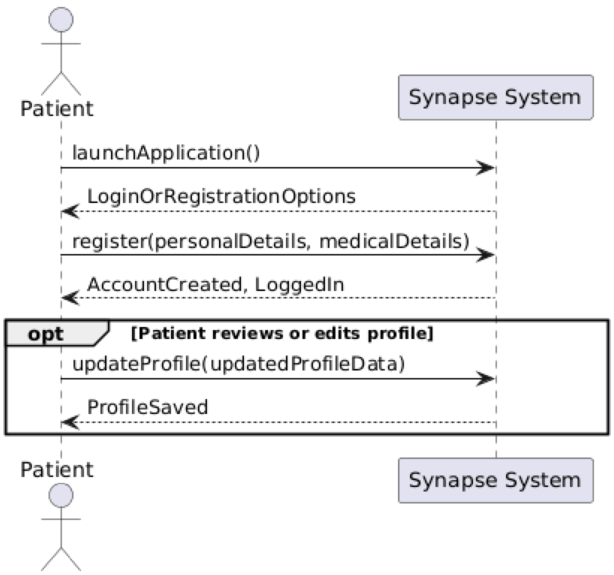
The user interface is a JavaFX-based graphical desktop application featuring distinct view screens for each major function, such as Main Dashboard, Medicine View, and Diet Hydration View. The interface employs a persistent navigation menu for seamless transitioning between modules. A highly visible, uniquely colored Emergency button is accessible from the main dashboard at all times. Standardized form inputs are used for capturing data, with built-in validation that restricts numeric fields from accepting alphabetic characters. System feedback and error messages are displayed dynamically, utilizing standard conventions such as changing text color to red to indicate format errors or failed submissions. Standardized pop-up dialogues are utilized for system alerts and interaction warnings.

● Domain Model


HydrationLog
- logId : UUID
- logDate : Date
- waterConsumedML : Integer
- dailyGoalMl : Integer
DietLog
- logId : UUID
- logDate : Date
- totalCalories : Integer
FoodItem
- itemId : UUID
- name : String
- portionSize : String
- estimatedCalories : Integer
Contains
1
1..*
VitalLog
- logId : UUID
- timestamp : DateTime
- bloodPressure : String
- heartRateBpm : Integer
- temperatureF : Double
- isAbnormal : Boolean
SymptomLog
- logId : UUID
- timestamp : DateTime
- symptomName : String
- severityLevel : Integer
- notes : String
WeightLog
- logId : UUID
- timestamp : DateTime
- weightKg : Double
- calculatedBMI : Double
SystemAlert
- alertId : UUID
- alertType : String
- message : String
- scheduledDateTime : DateTime
- isAcknowledged : Boolean
Triggers
logs
Tracks
Tracks
logs
Receives
Tracks
Triggers
Triggers
0..*
0..1
1
Patient
- patientId : UUID
- fullName : String
- email : String
- passwordHash : String
- dateOfBirth : Date
- gender : String
- heightCm : Double
- currentBMI : Double
CalendarEvent
- eventId : UUID
- title : String
- eventType : String
- eventDate : Date
- eventTime : Time
- reminderPreferences : String
MedicationSchedule
- scheduleId : UUID
- dosageAmount : String
- scheduledTime : Time
- frequency : String
- isPrescribed : Boolean
Schedules
Tracks
has
Medicine
- medicineId : UUID
- name : String
- inventoryQuantity : Integer
- expiryDate : Date
is involved in
DrugInteraction
- interactionId : UUID
- severityLevel : String
- warningMessage : String
EmergencyProfile
- profileId : UUID
- bloodType : String
- allergies : String
- chronicConditions : String
- emergencyContactName : String
- emergencyContactPhone : String
Generates
Creates
Writes
Browses
Organizes
HealthcareFacility
- facilityId : UUID
- name : String
- facilityType : String
- address : String
- contactNumber : String
- specialty : String
- services : String
HealthReport
- reportId : UUID
- generationDate : DateTime
- dateRangeStart : Date
- dateRangeEnd : Date
- includedCategories : String
- pdfFilePath : String
MedicalRecord
- recordId : UUID
- title : String
- category : String
- fileFormat : String
- fileSize : Double
- filePath : String
- uploadDate : Date
DataBackup
- backupId : UUID
- creationDate : DateTime
- filePath : String
- fileSize : Double
- status : String
JournalEntry
- entryId : UUID
- timestamp : DateTime
- contentText : String
- moodTag : String

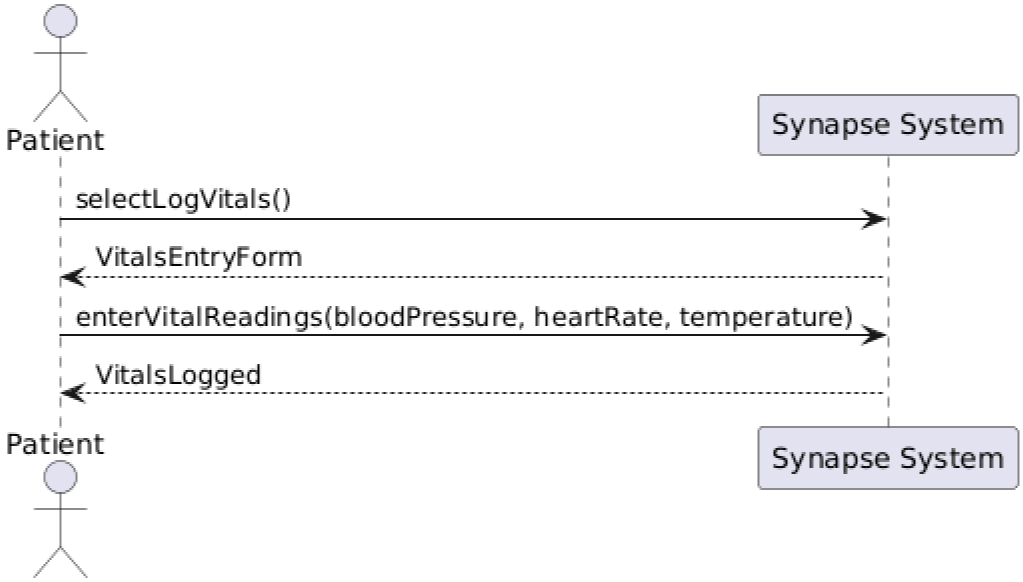
- # System Sequence Diagram

Use Case: Manage Account



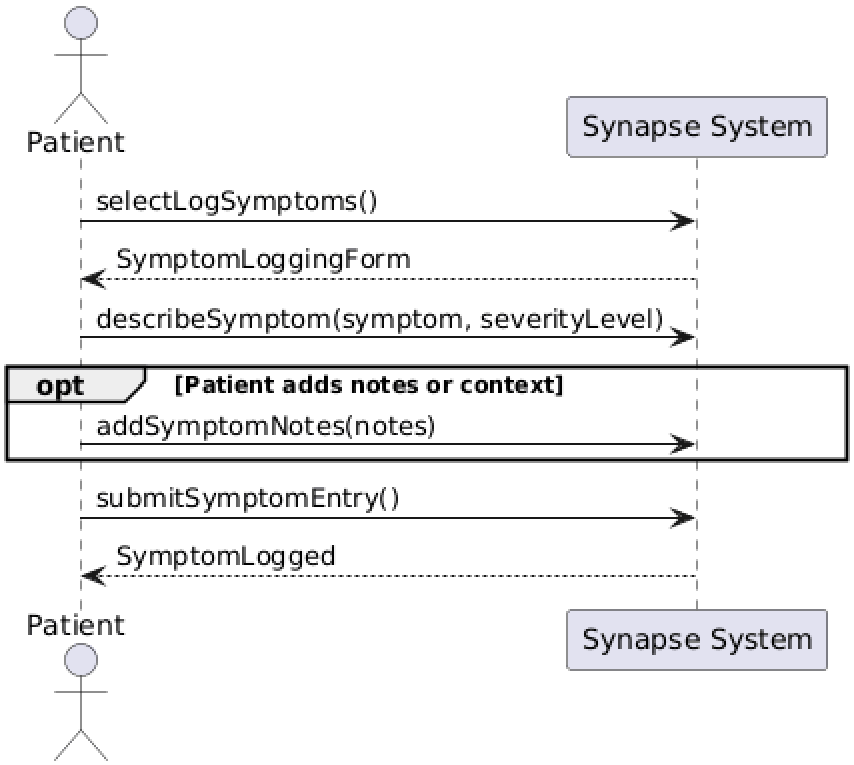
UC-02

Use Case: Log Vitals



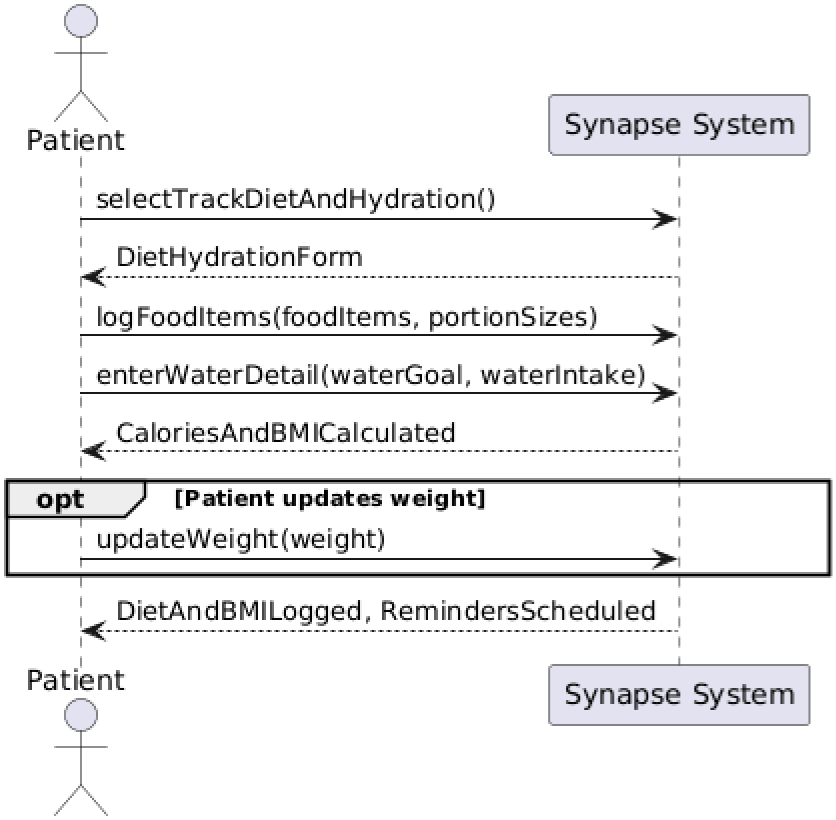
UC-03

Use Case: Log Symptoms



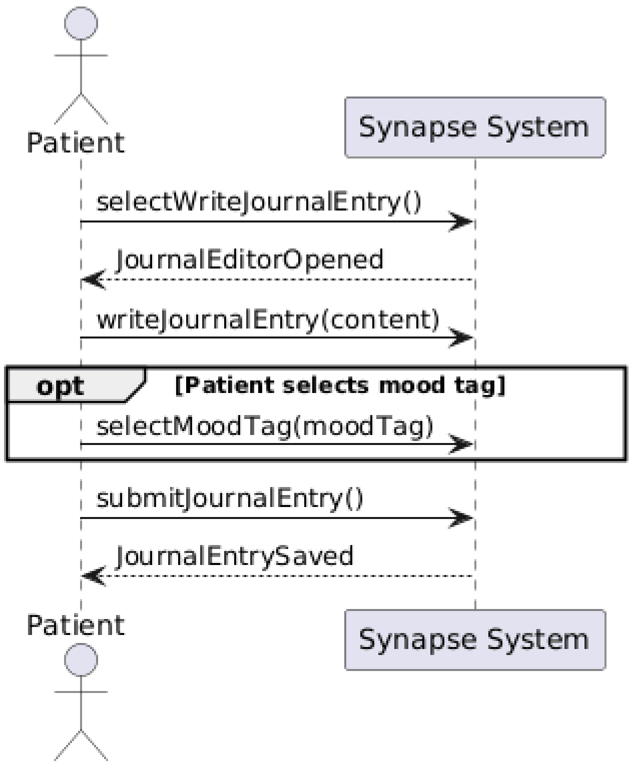
UC-04

Use Case: Track Diet, BMI and Hydration



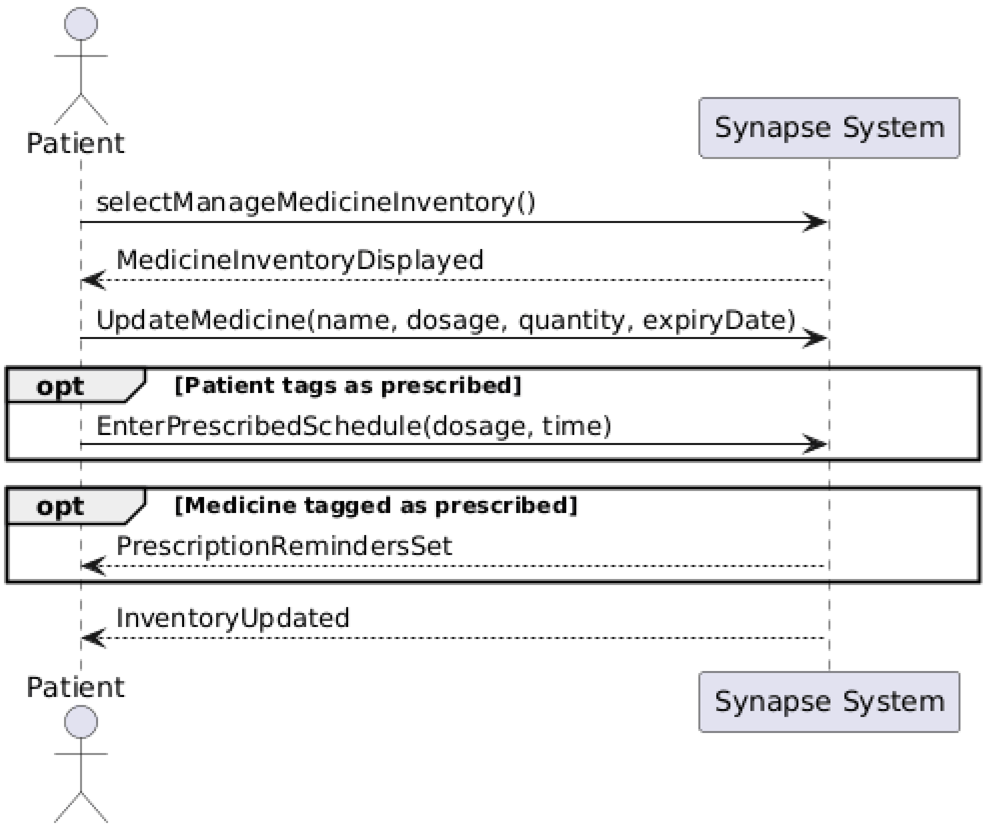
UC-05

Use Case: Write Journal Entry



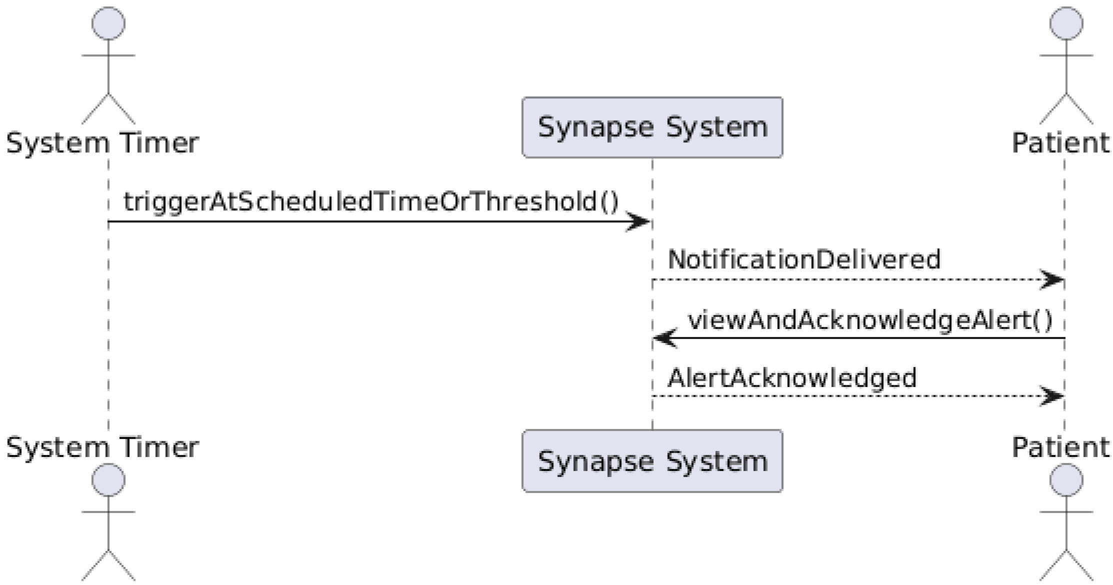
UC-06

Use Case: Manage Medicine Inventory & Prescriptions



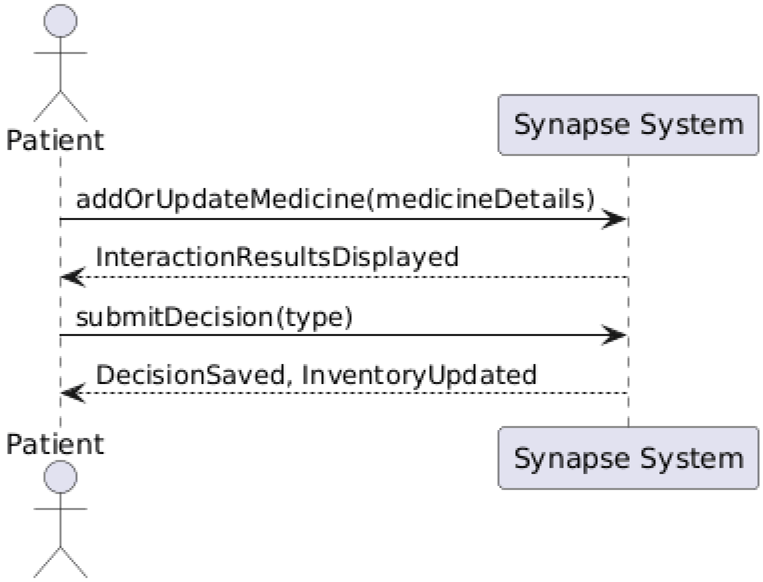
UC-07

Use Case: Send System Alert



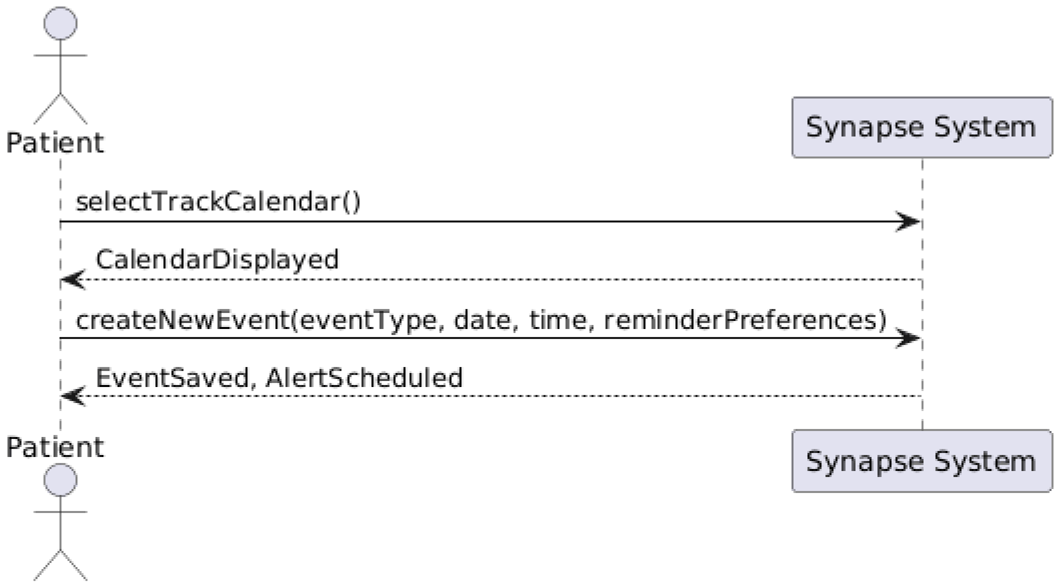
UC-08

Use Case: Check Drug Interaction



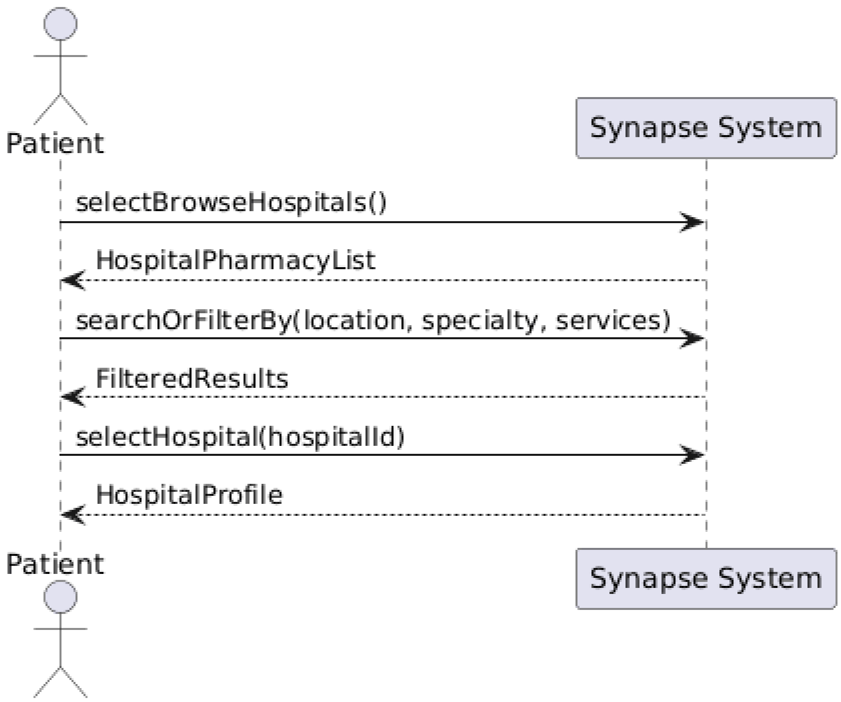
UC-09

Use Case: Track Calendar



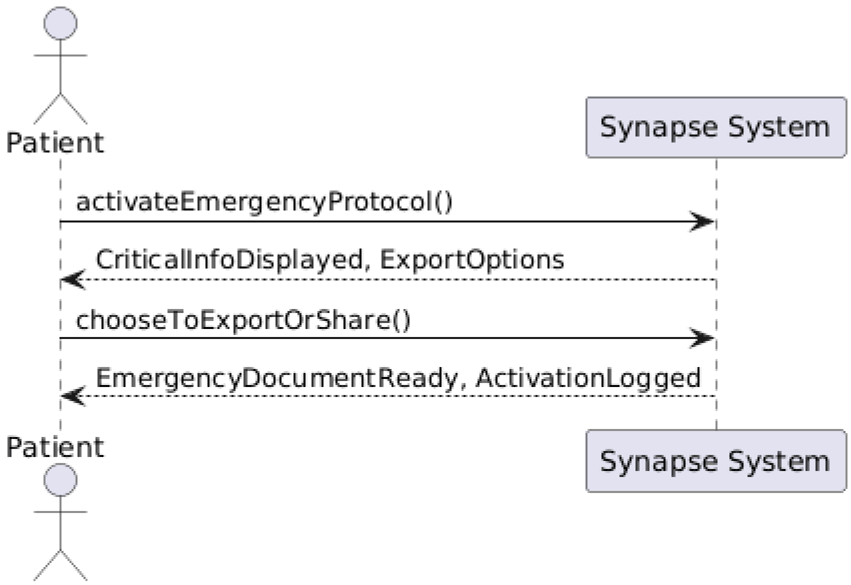
UC-10

Use Case: Browse Hospitals



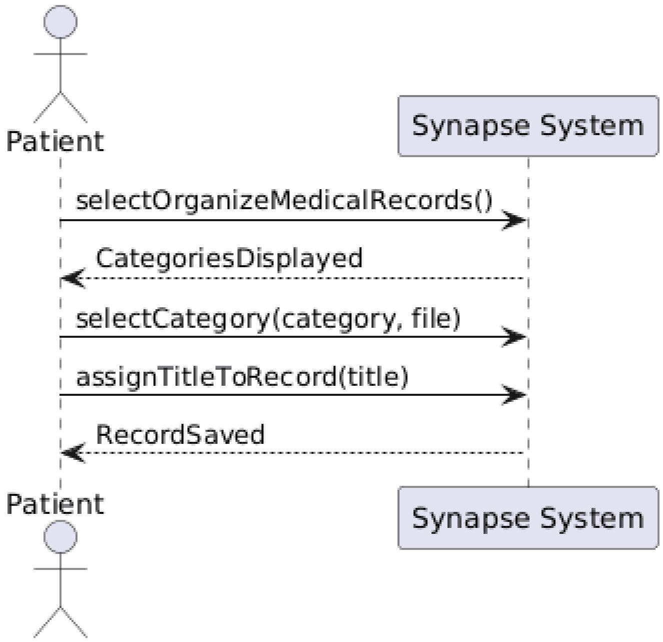
UC-11

Use Case: Trigger Emergency Protocol



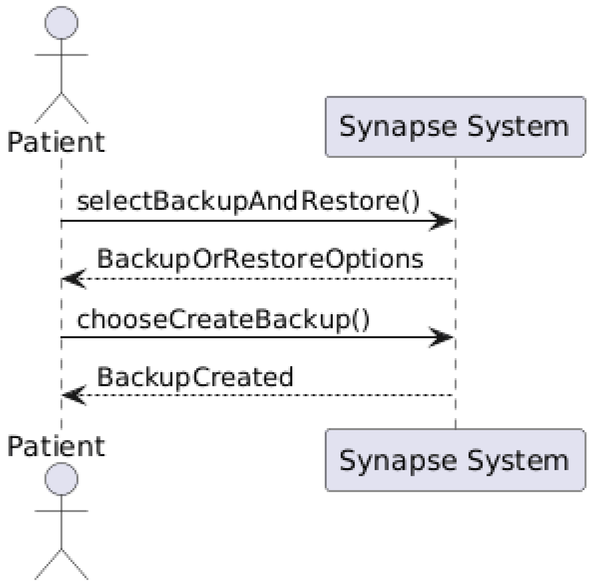
UC-12

Use Case: Organize Medical Records



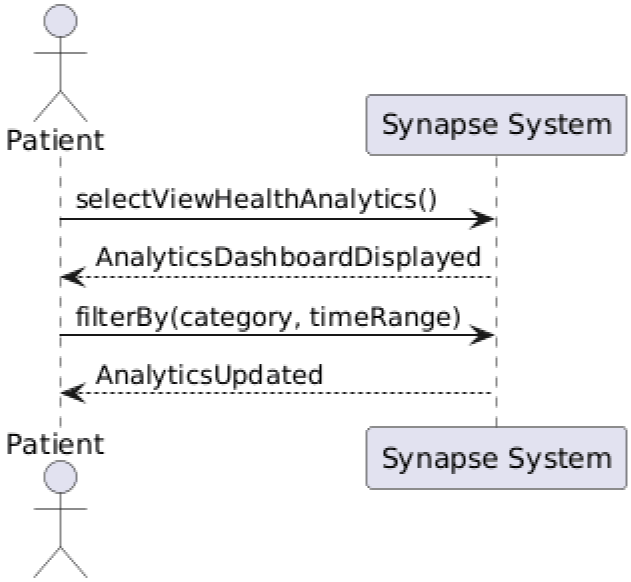
UC-13

Use Case: Backup and Restore Data



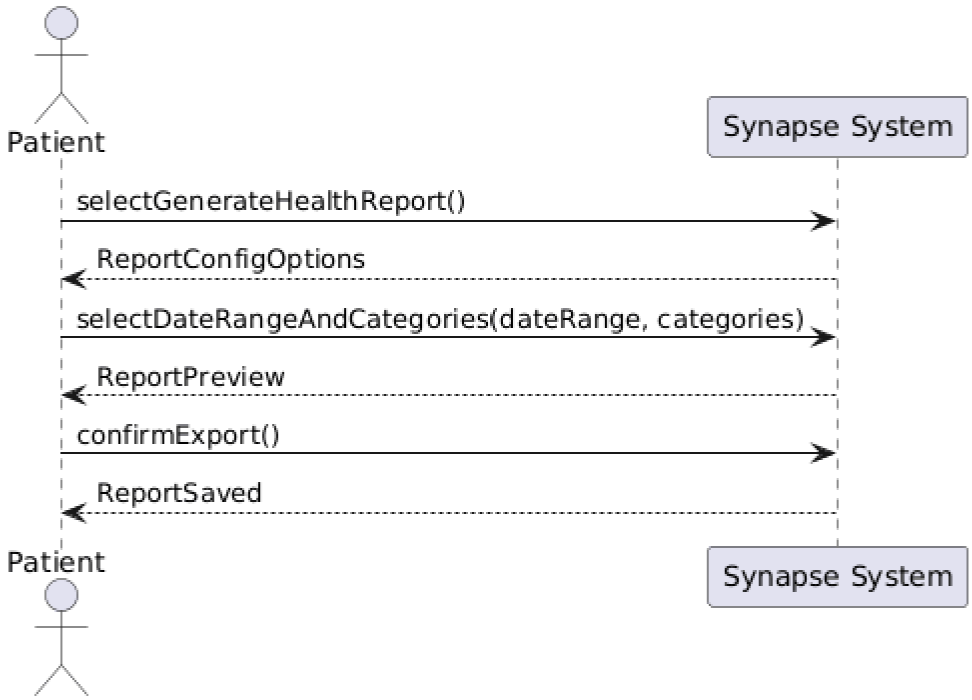
UC-14

Use Case: View Health Analytics



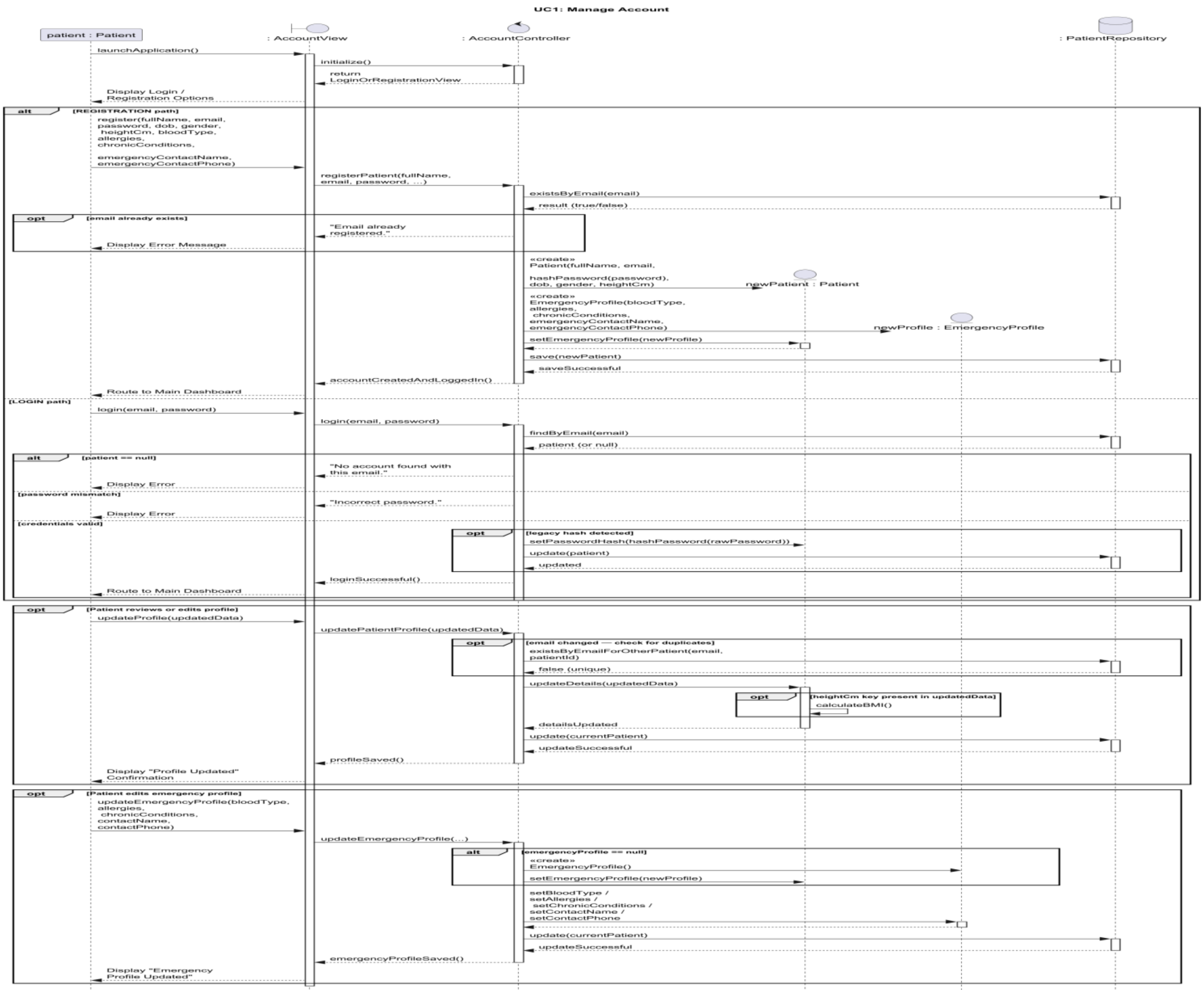
UC-15

Use Case: Generate Health Report



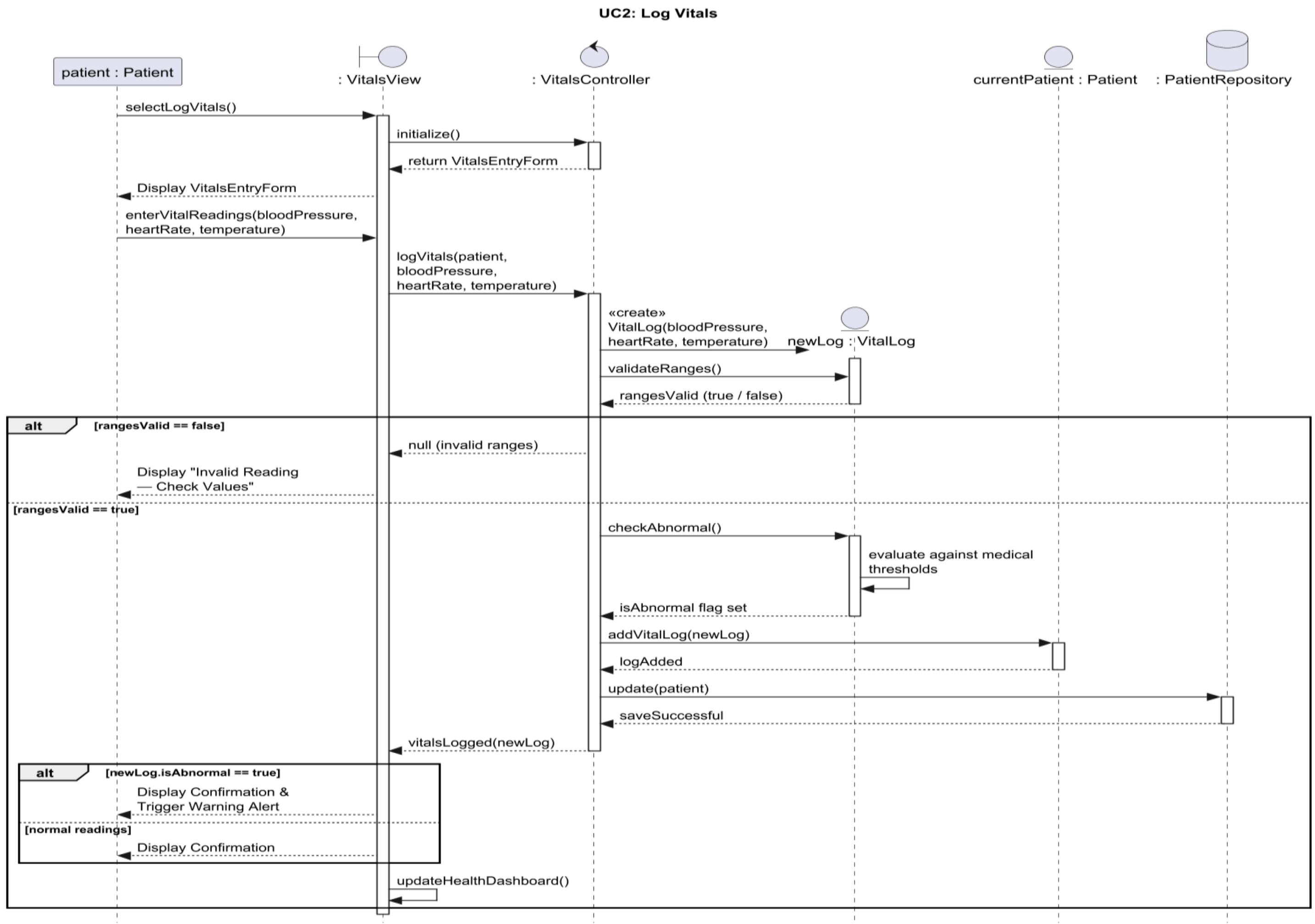
● **Sequence Diagram**

Use Case: Manage Account



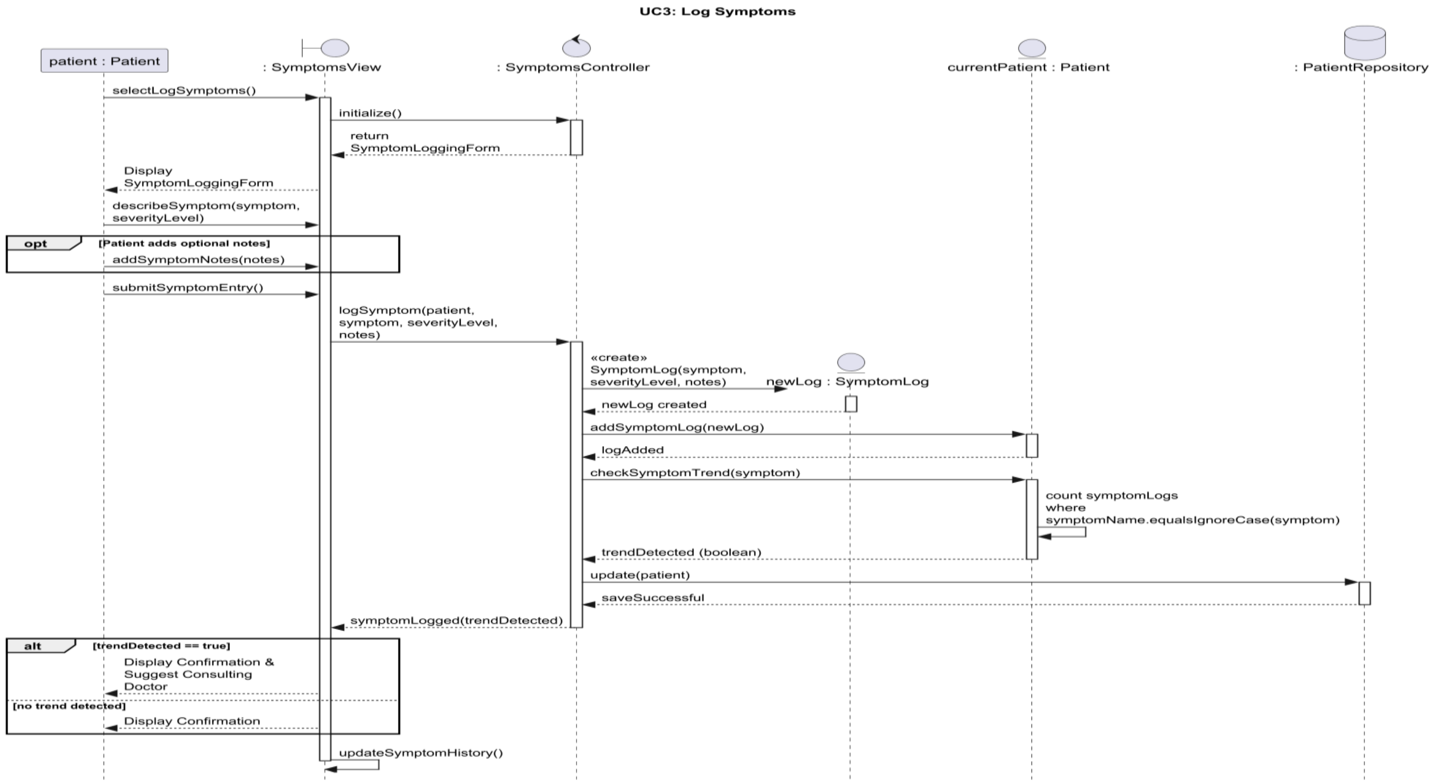
UC-02

Use Case: Log Vitals



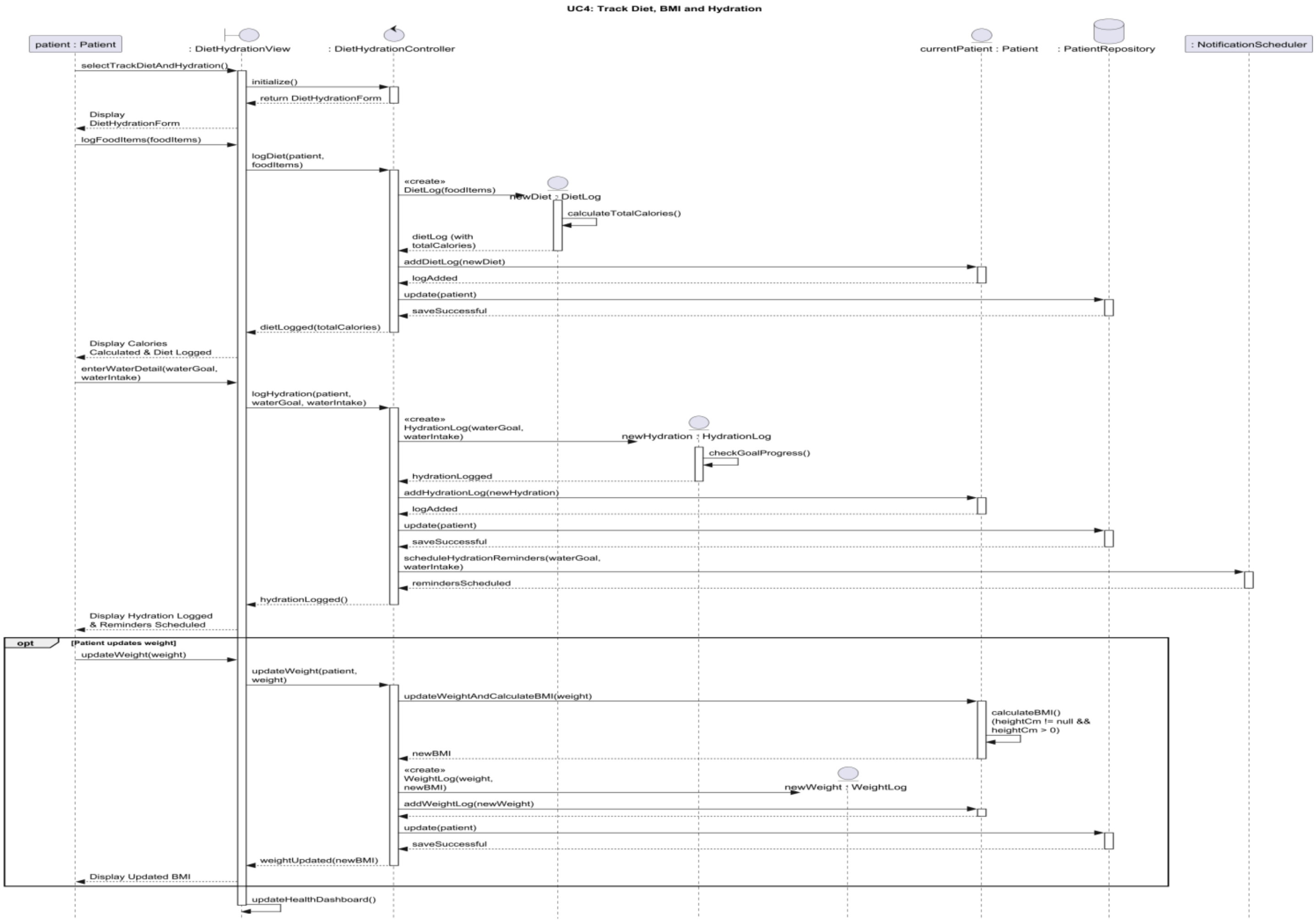
UC-03

Use Case: Log Symptoms



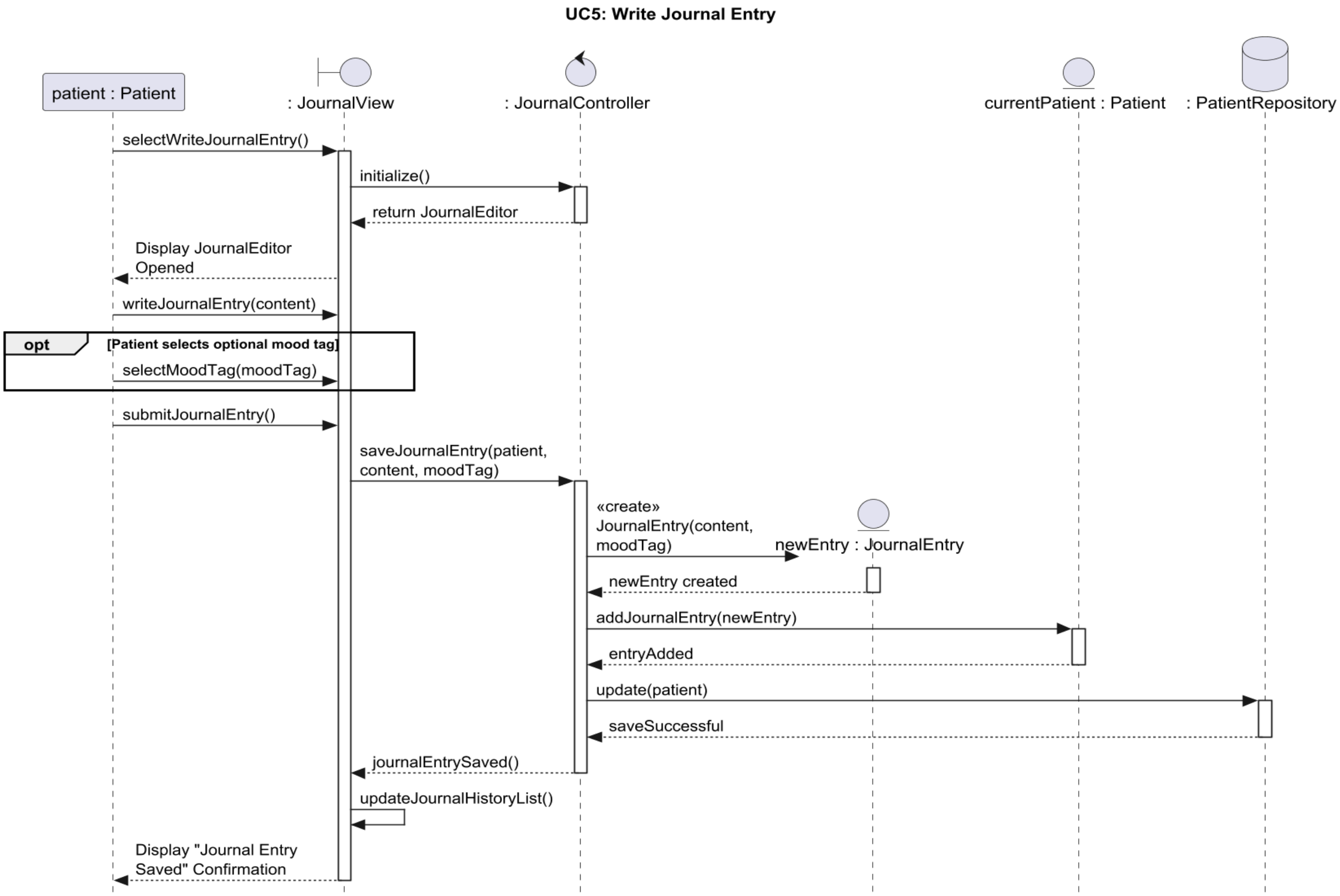
UC-04

Use Case: Track Diet, BMI and Hydration



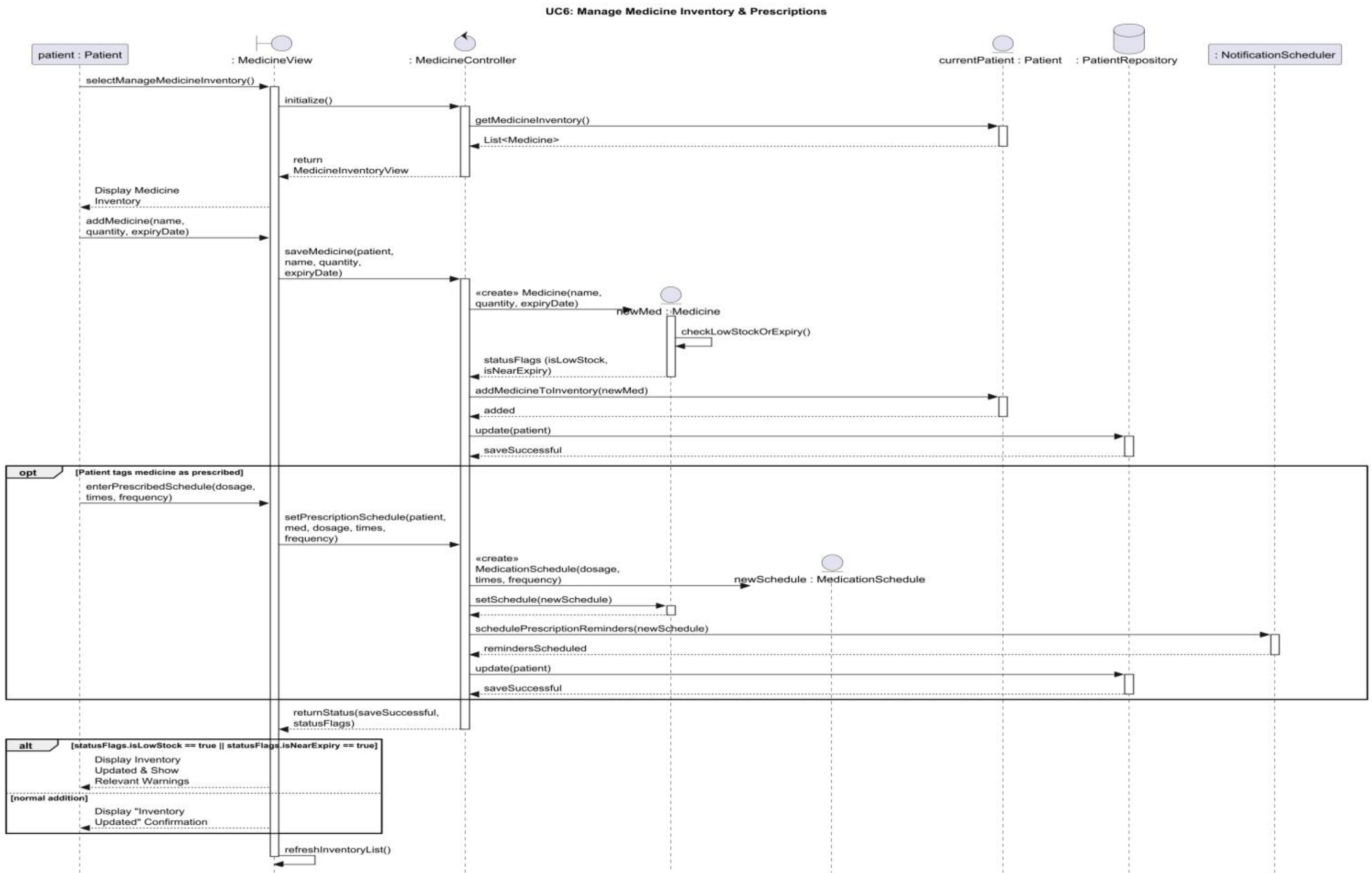
UC-05

Use Case: Write Journal Entry



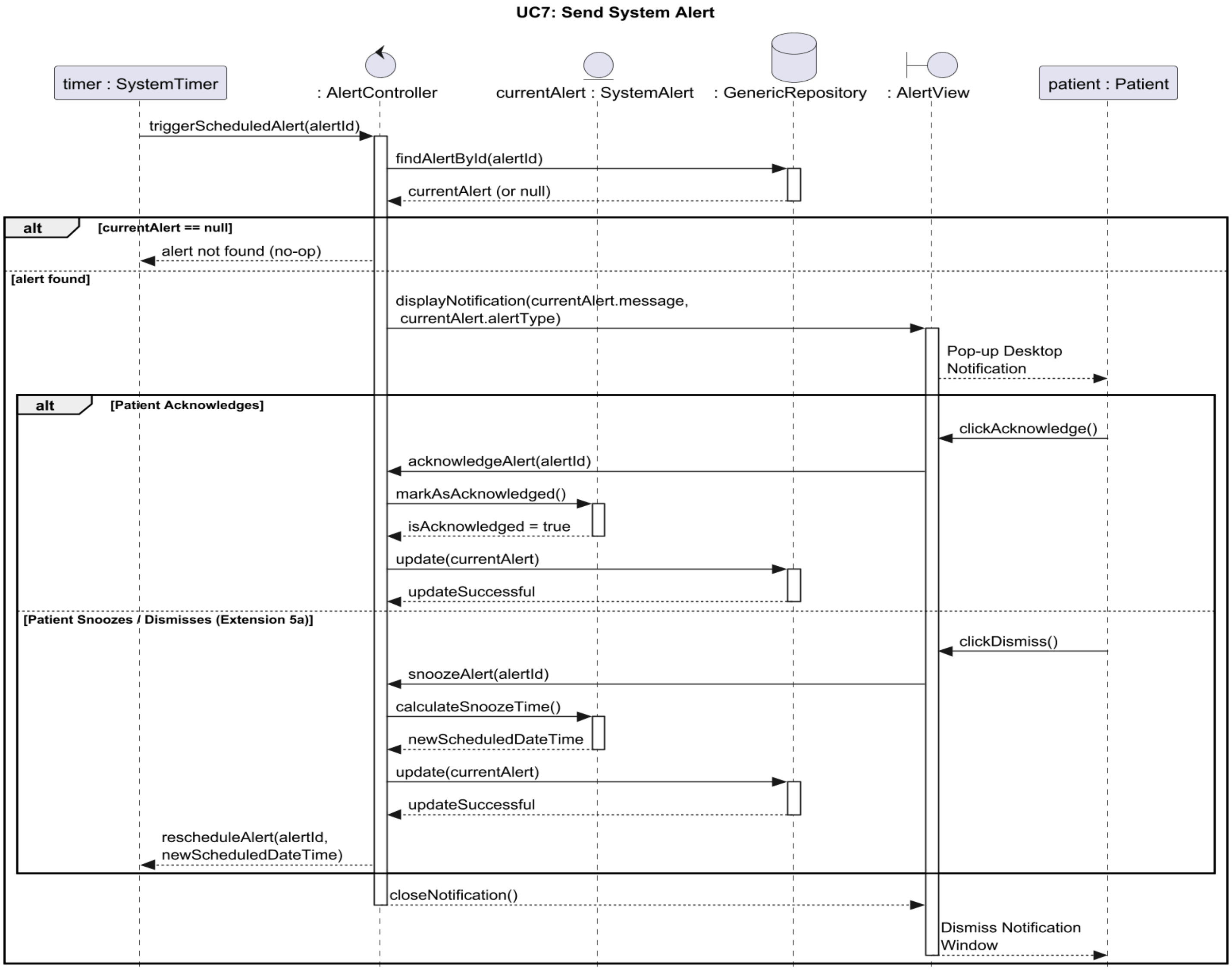
UC-06

Use Case: Manage Medicine Inventory & Prescriptions



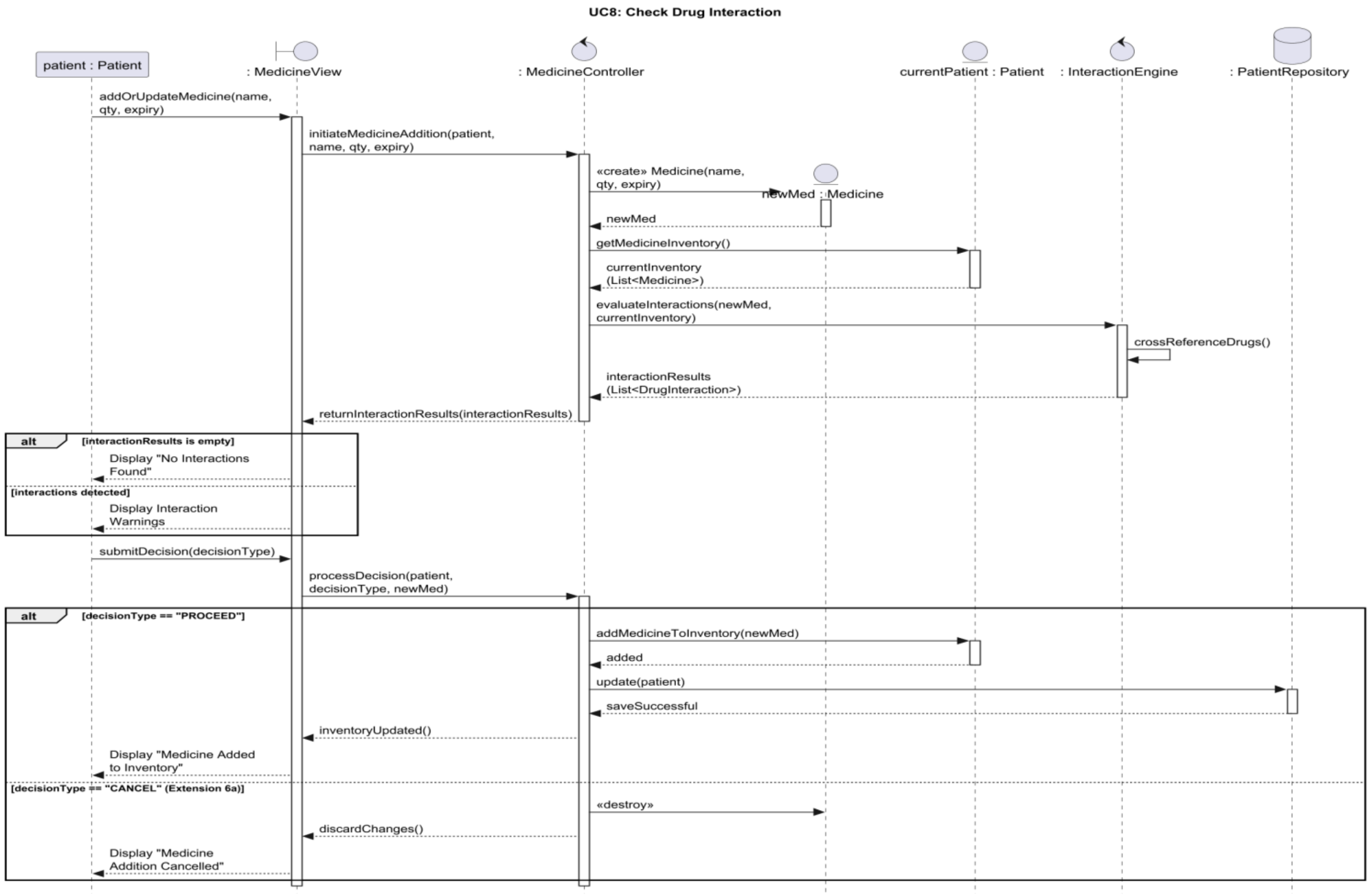
UC-07

Use Case: Send System Alert



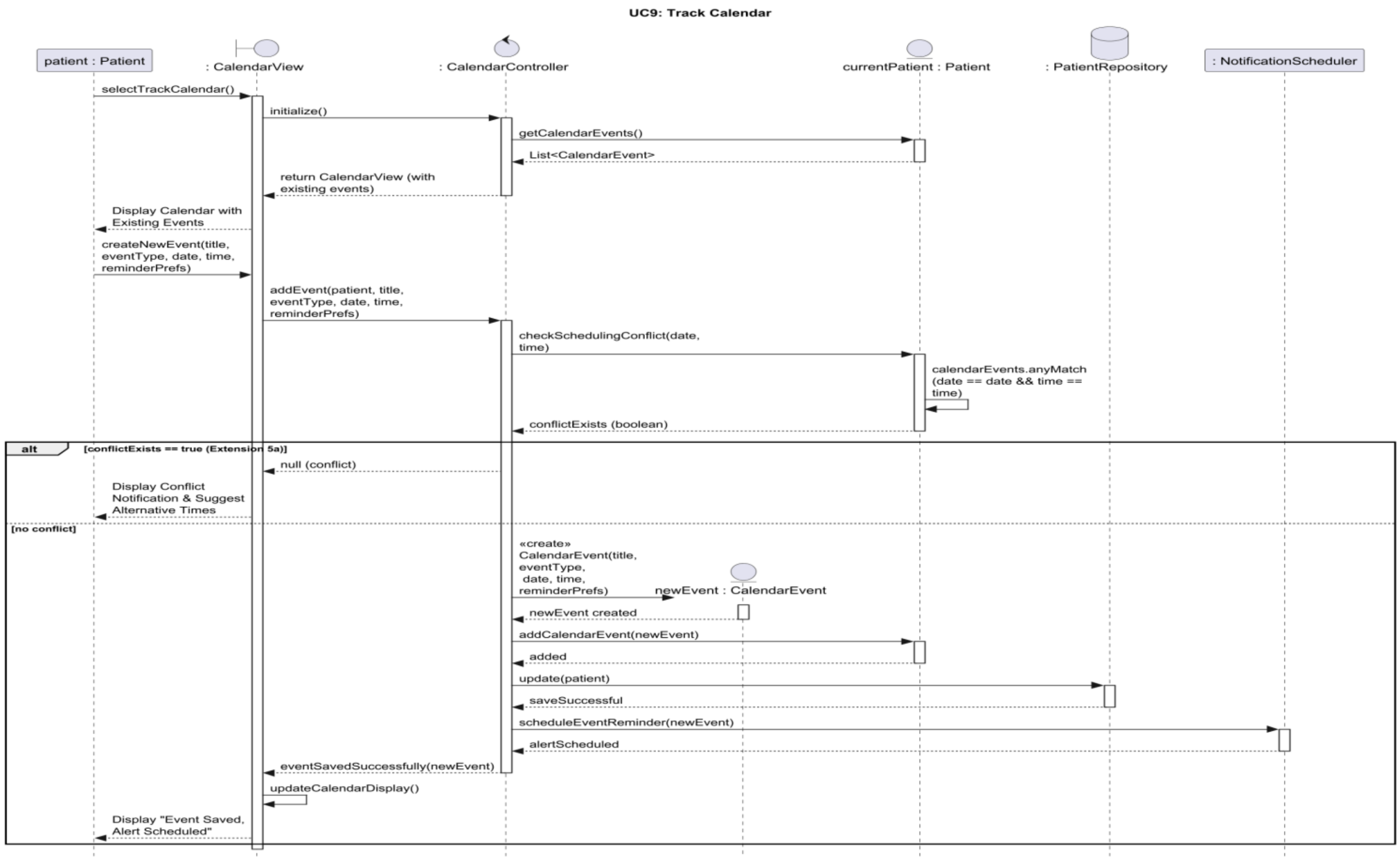
UC-08

Use Case: Check Drug Interaction

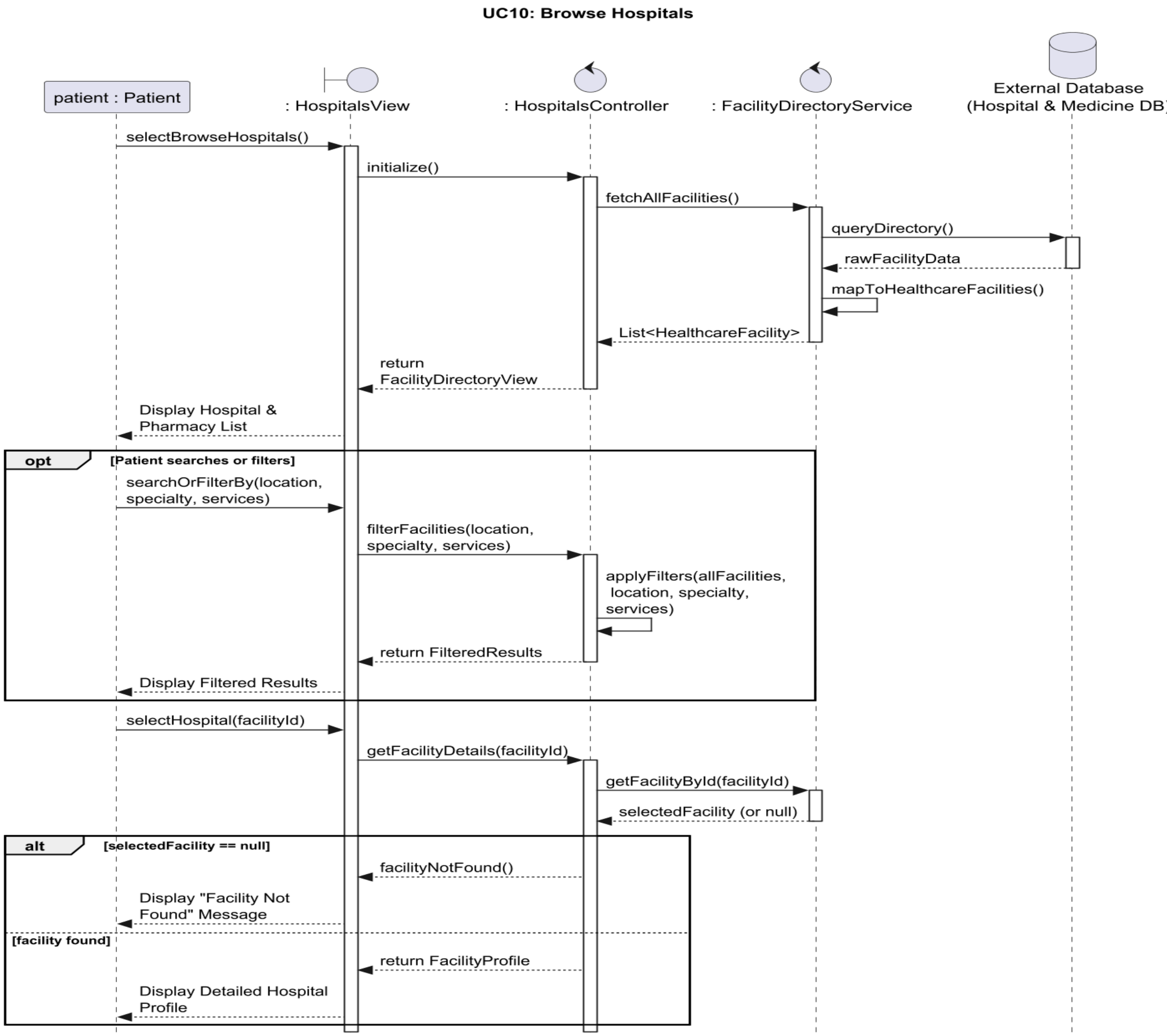


UC-09

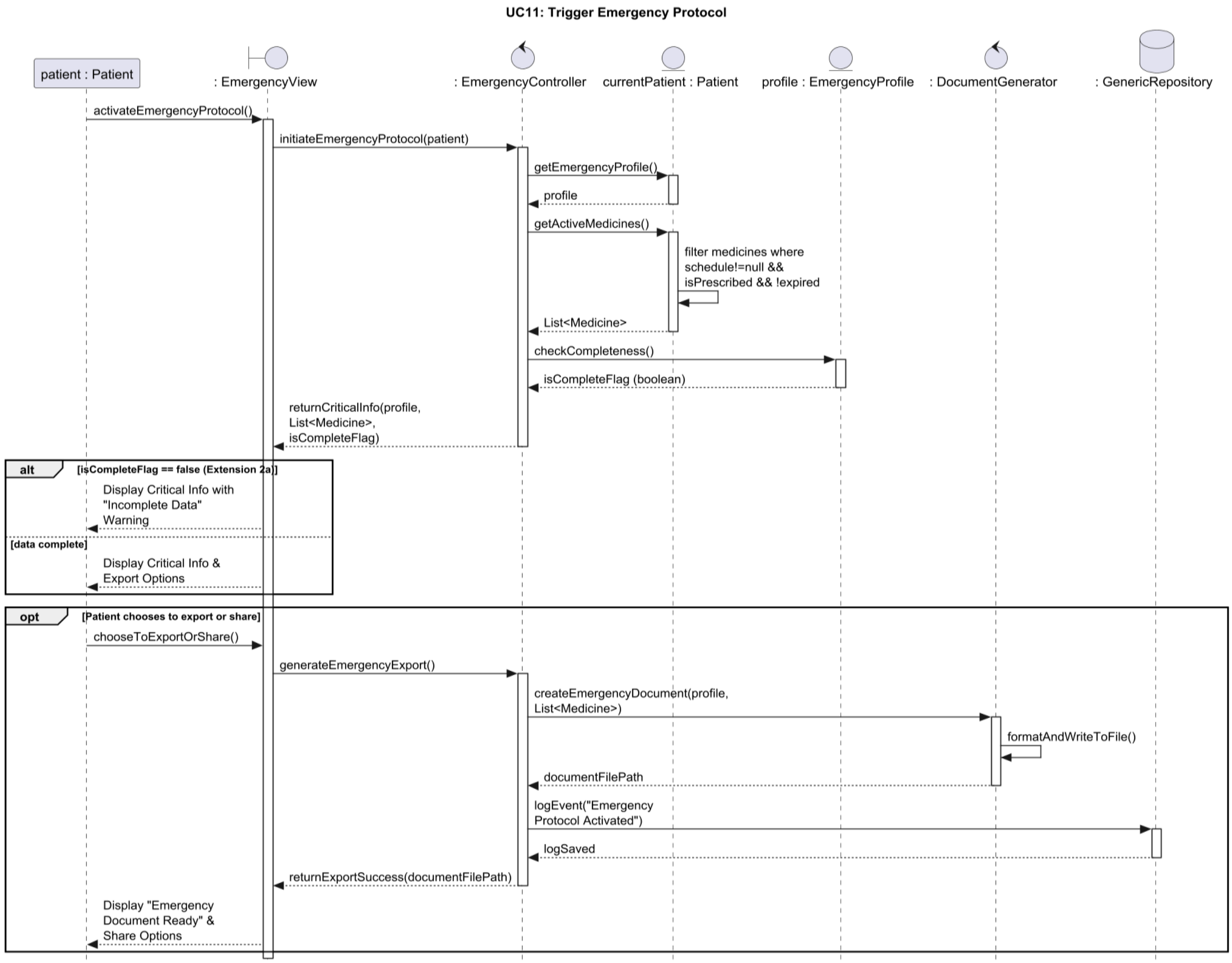
Use Case: Track Calendar



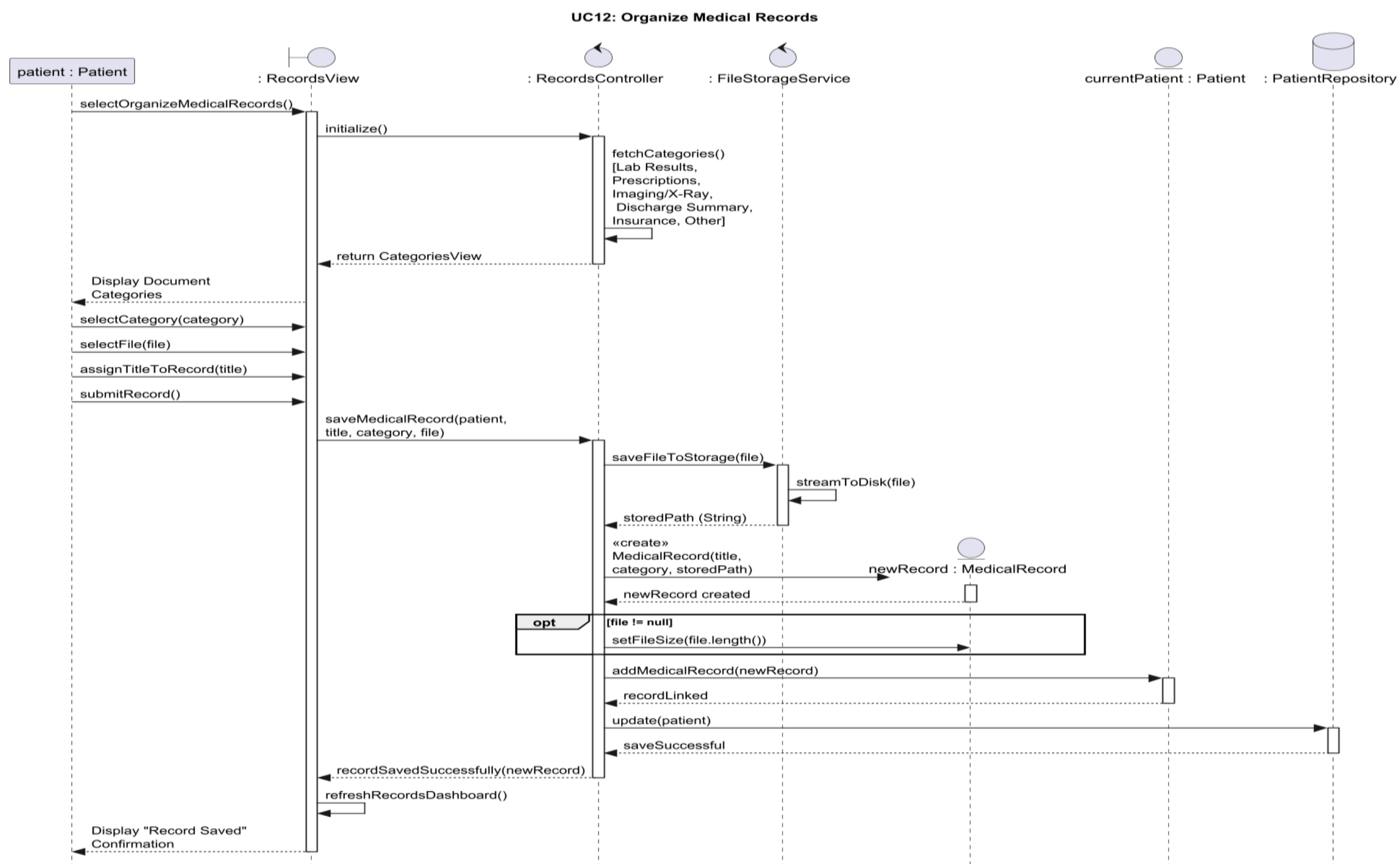
Use Case: Browse Hospitals



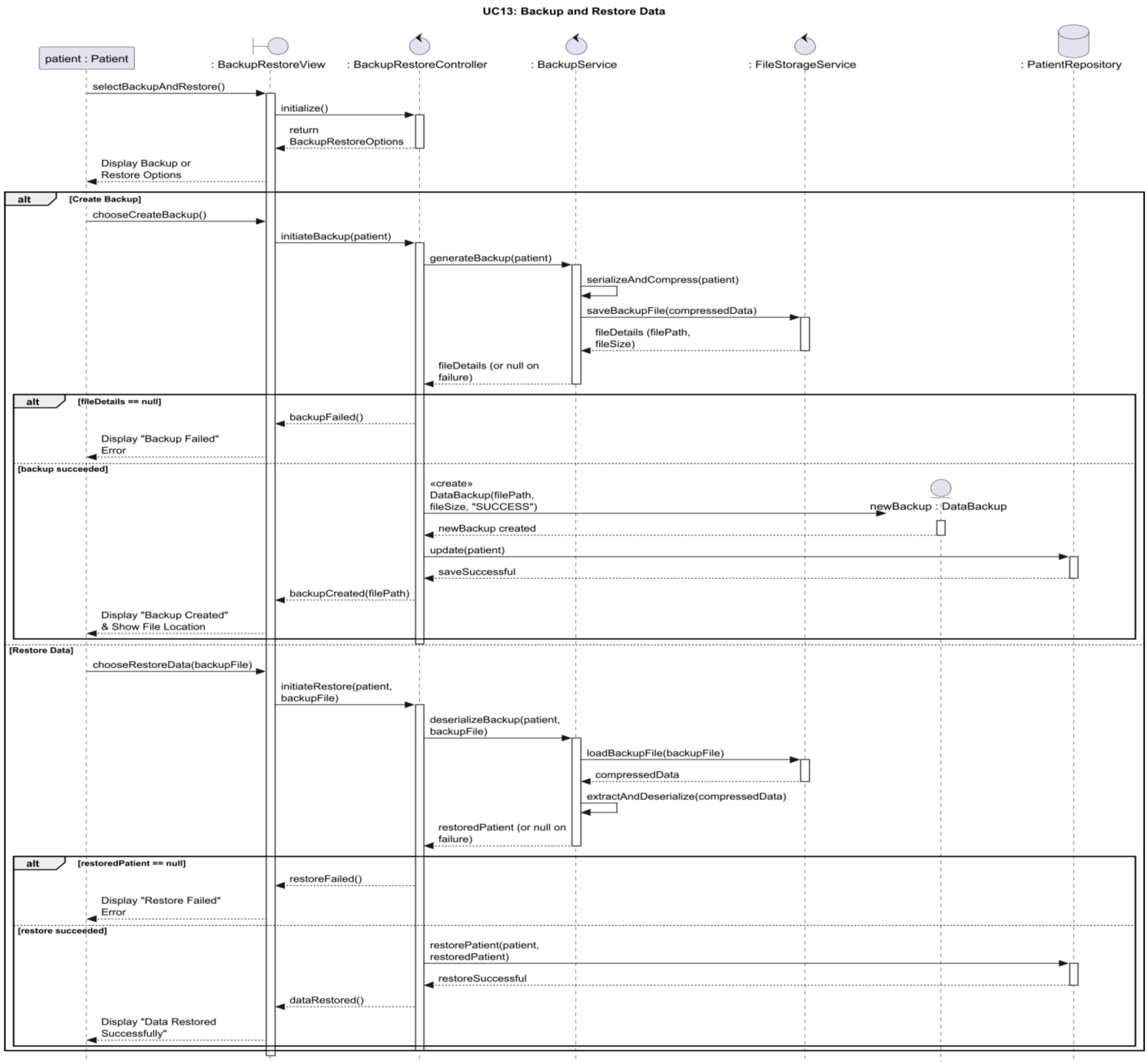
Use Case: Trigger Emergency Protocol



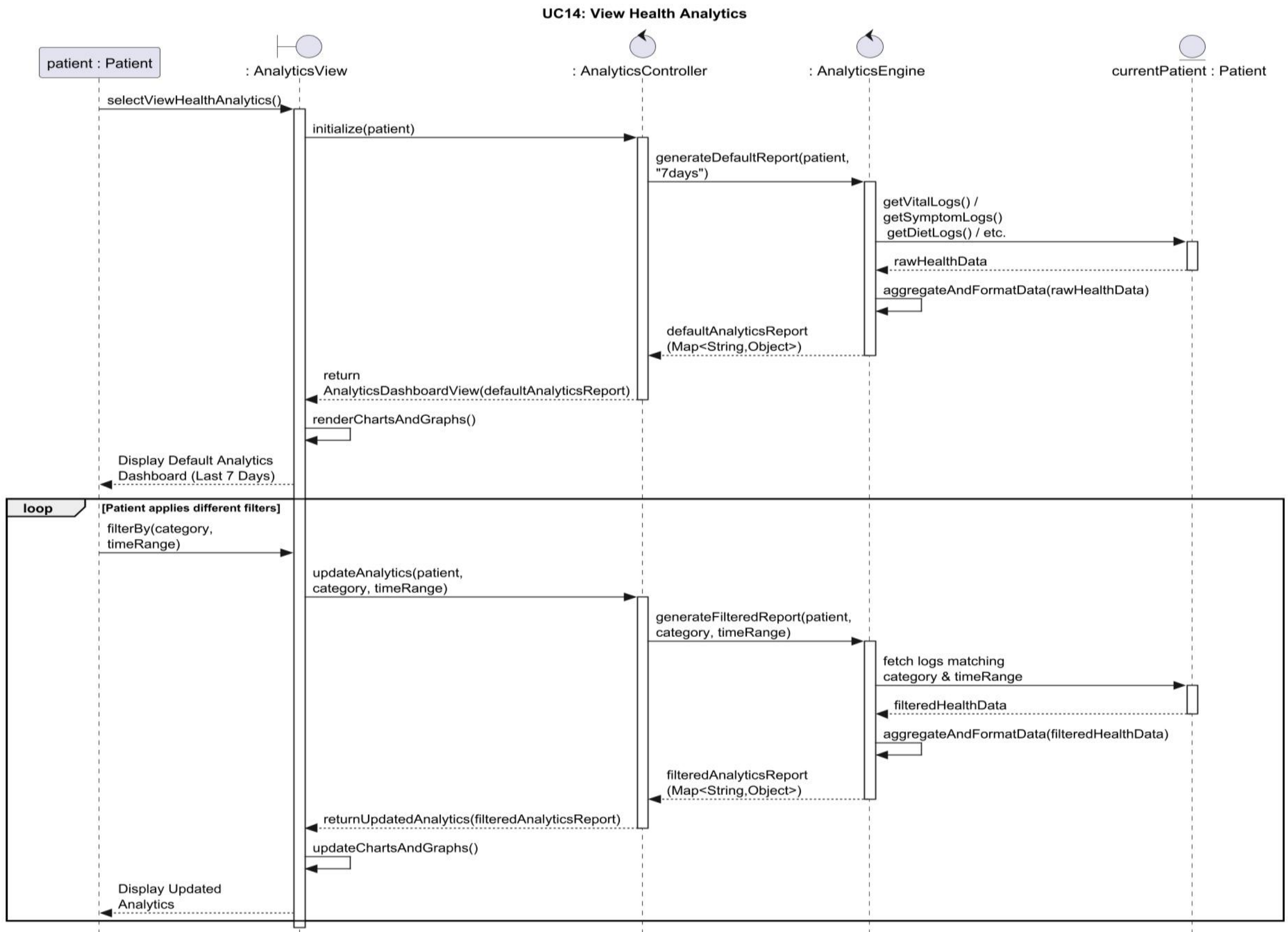
Use Case: Organize Medical Records



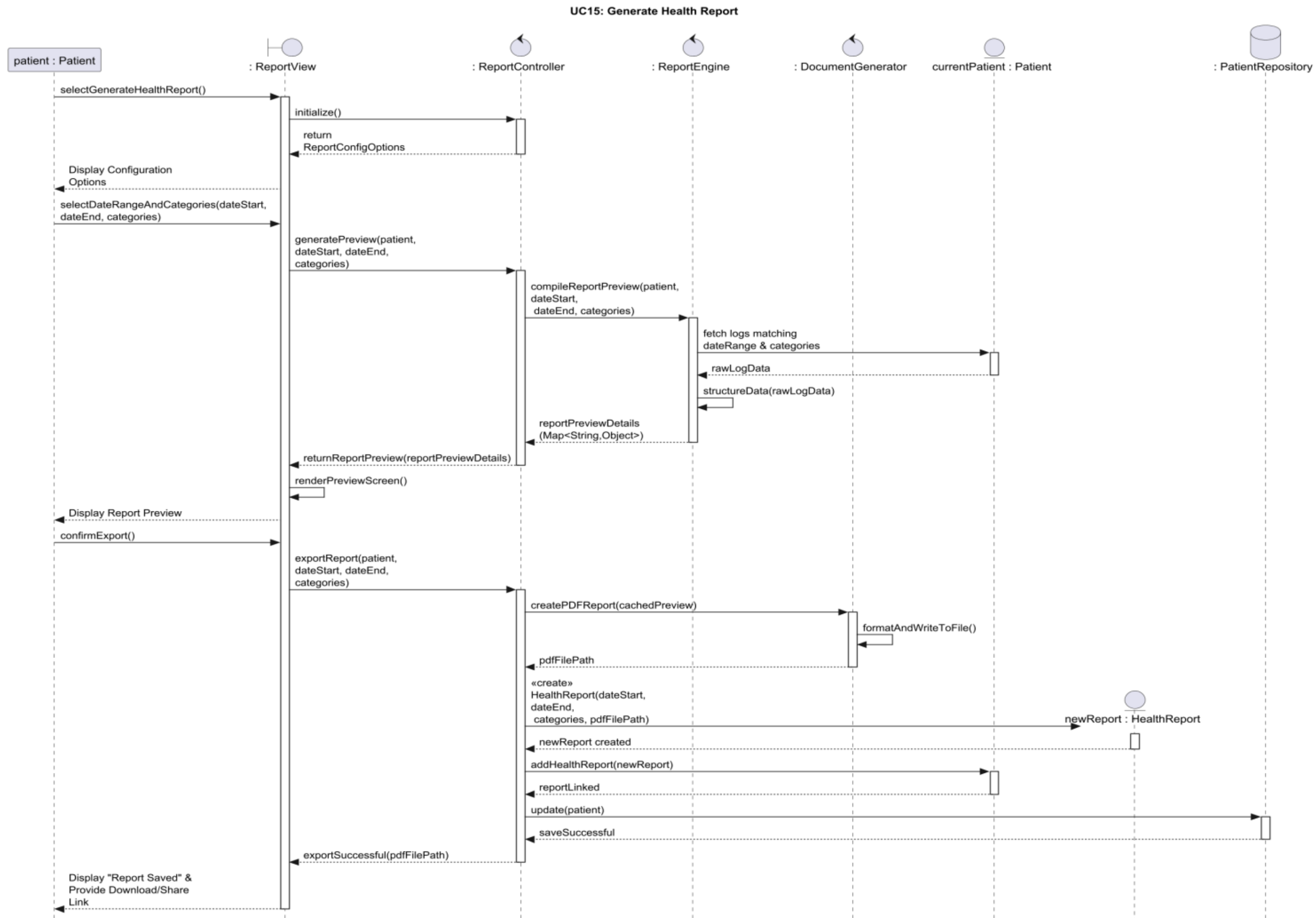
Use Case: Backup and Restore Data



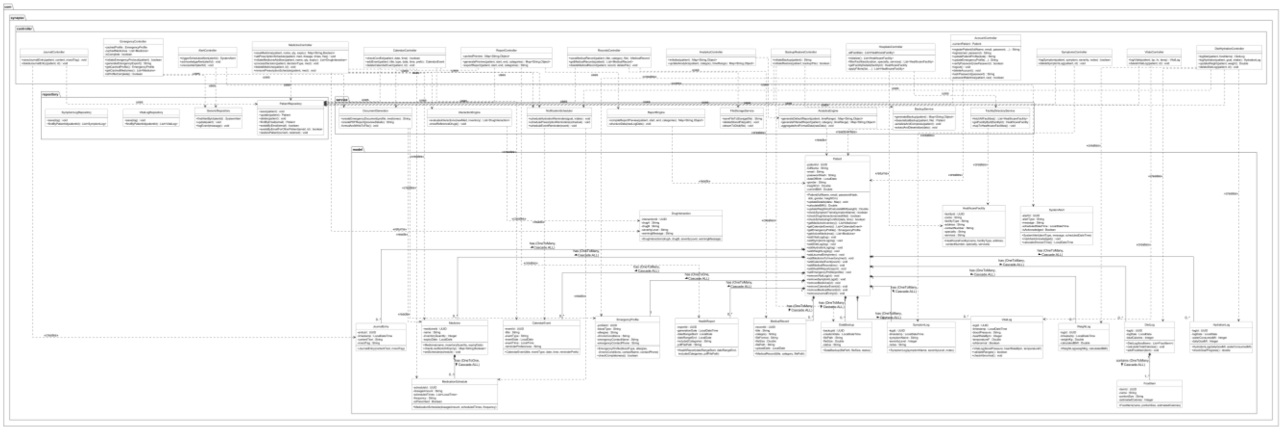
Use Case: View Health Analytics



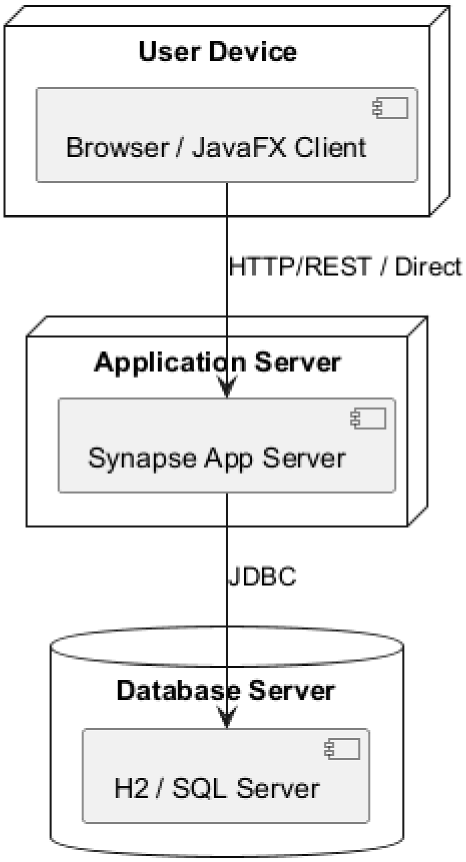
Use Case: Generate Health Report



● Class Diagram

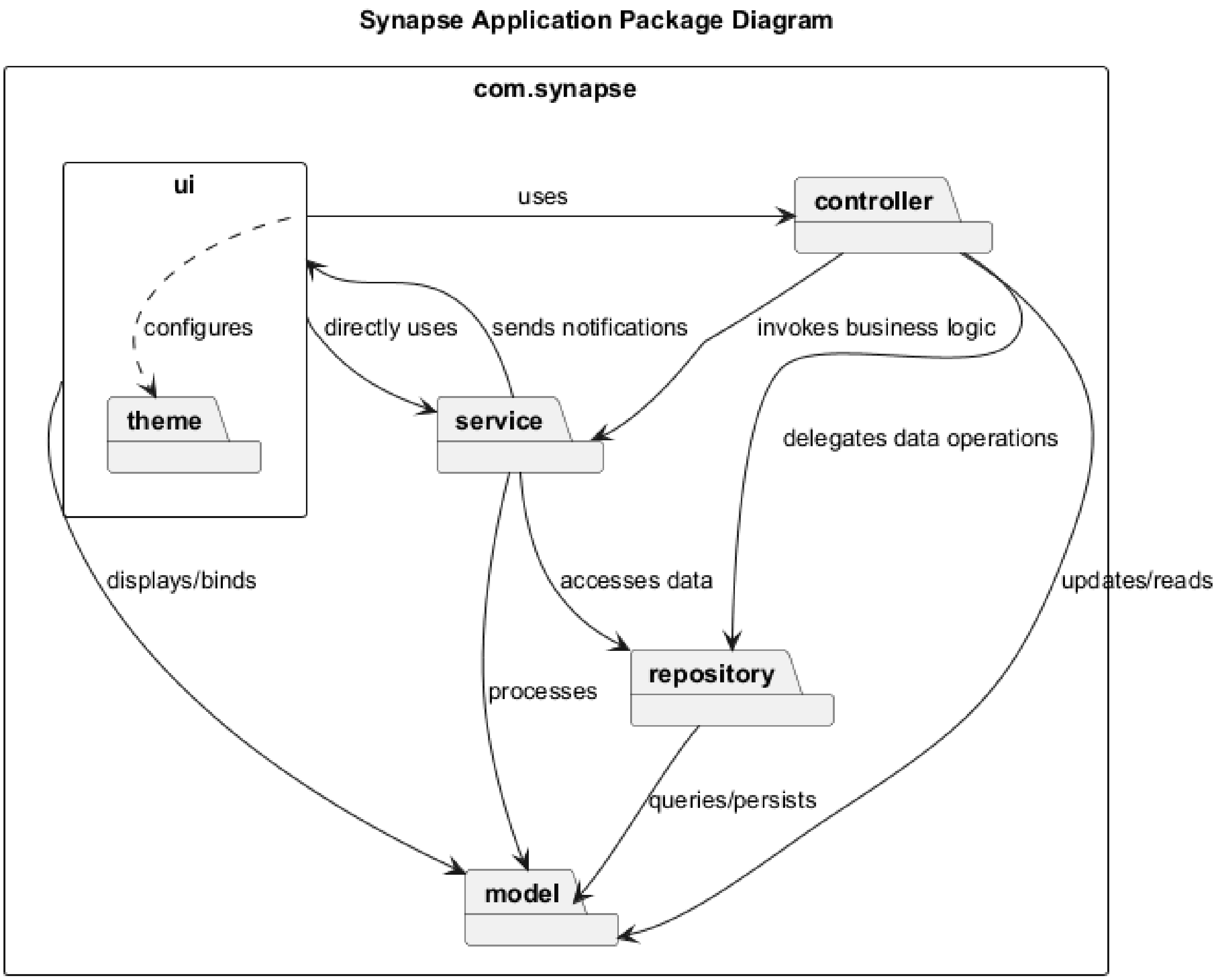


- **Deployment Diagram**



● Package Diagram

```
Please use '!option handwritten true' to enable handwritten
```



● System SD Workflow